



Progressive Education Society's
Modern College of Engineering, Pune - 411005
Department of Electronics & Computer

AY-2024-25 Semester - I

Microcontroller and Applications

Course In-charge
Mrs. R. S. Chaudhari

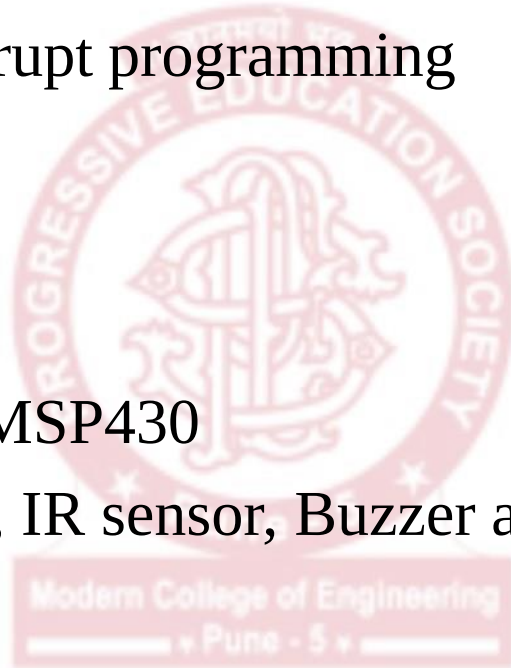


Unit V

Real World Interfacing

Syllabus

- GPIO programming
- I/O multiplexing
- Interrupts and interrupt programming
- Timers
- Watchdog timer
- PWM control
- ADC and DAC in MSP430
- Interfacing of LED, IR sensor, Buzzer and Relay
- UART protocol
- I2C protocol
- SPI protocol



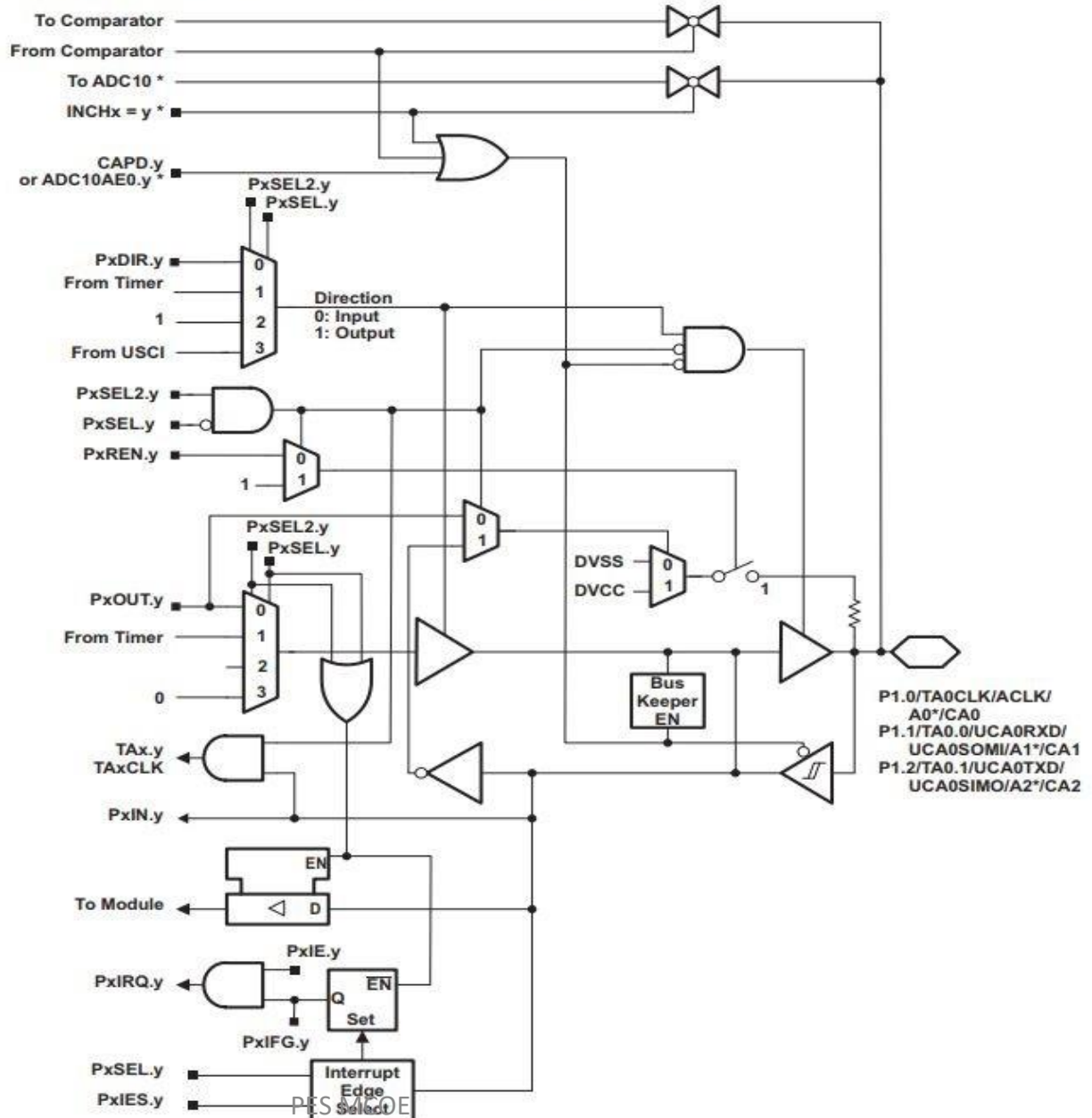
Digital I/O

- MSP430G2553 (20 pin) has two Ports P1 and P2.
- MSP430G2553 (28 pin) has three ports P1, P2, P3.
- Each Port has 8 I/O pins.
- Any I/O can be made input or output, and can be individually read or written to.
- Individually configurable interrupts on P1 and P2.
- Choice of individually configuring pull-up or pull down resistors.
- The ports are configured by several 8 bit Peripheral Registers.

Port Schematics

PORT SCHEMATICS

Port P1 Pin Schematic: P1.0 to P1.2, Input/Output With Schmitt Trigger



* Note: MSP430G2x53 devices only. MSP430G2x13 devices have no ADC10.

Port Schematics

Settings for selecting the function of a given pin is provided in respective datasheet. Example is provided here:-

Table 16. Port P1 (P1.0 to P1.2) Pin Functions

PIN NAME (P1.x)	x	FUNCTION	CONTROL BITS AND SIGNALS ⁽¹⁾				
			P1DIR.x	P1SEL.x	P1SEL2.x	ADC10AE.x INCH.x=1 ⁽²⁾	CAPD.y
P1.0/ TA0CLK/ ACLK/ A0 ⁽²⁾ / CA0/ Pin Osc	0	P1.x (I/O)	I: 0; O: 1	0	0	0	0
		TA0.TACLK	0	1	0	0	0
		ACLK	1	1	0	0	0
		A0	X	X	X	1 (y = 0)	0
		CA0	X	X	X	0	1 (y = 0)
		Capacitive sensing	X	0	1	0	0
P1.1/ TA0.0/ UCA0RXD/ UCA0SOMI/ A1 ⁽²⁾ / CA1/ Pin Osc	1	P1.x (I/O)	I: 0; O: 1	0	0	0	0
		TA0.0	1	1	0	0	0
		TA0.CCI0A	0	1	0	0	0
		UCA0RXD	from USCI	1	1	0	0
		UCA0SOMI	from USCI	1	1	0	0
		A1	X	X	X	1 (y = 1)	0
		CA1	X	X	X	0	1 (y = 1)
		Capacitive sensing	X	0	1	0	0
P1.2/ TA0.1/ UCA0TXD/ UCA0SIMO/ A2 ⁽²⁾ / CA2/ Pin Osc	2	P1.x (I/O)	I: 0; O: 1	0	0	0	0
		TA0.1	1	1	0	0	0
		TA0.CCI1A	0	1	0	0	0
		UCA0TXD	from USCI	1	1	0	0
		UCA0SIMO	from USCI	1	1	0	0
		A2	X	X	X	1 (y = 2)	0
		CA2	X	X	X	0	1 (y = 2)
		Capacitive sensing	X	0	1	0	0

(1) X = don't care

(2) MSP430G2x53 devices only

Digital I/O Registers

The MSP430 communicates with the Digital I/O Peripherals through a set of 8 bit Peripheral Registers:

- PxDIR
- PxIN
- PxOUT
- PxREN
- PxSEL and PxSEL2



PxDIR

- 'P' stands for port
- 'x' stands for port number (Upto 3 ports available on MSP430G2553, based on package selected)
- It is an eight bit register. It is used to define whether a pin is to be used as an input or as an output.
- If value of a bit in PxDIR register is '0', it is set as input.
- If value of a bit in PxDIR register is '1', it is set as output.
- Default value of PxDIR register for all ports is '0'.

PxIN

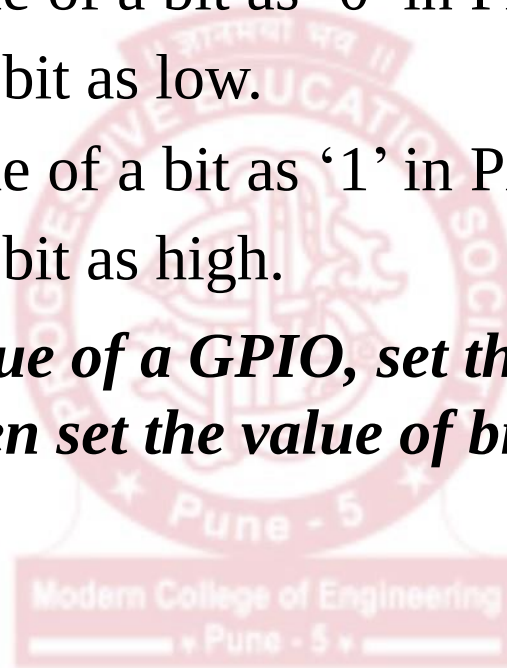
- This eight bit register stores the value of input on a port. Based on the value of logic low and logic high for MSP430 MCU, the voltage value read on the input pin will be classified as logic low or logic high.
- The '0' value of a bit in PxIN register indicates the input value of bit as low.
- The '1' value of a bit in PxIN register indicates the input value of bit as high.

For reading the value of a GPIO, set the pin as input using PxDIR register then read the value of bit in PxIN register.

PxOUT

- This eight bit register sets the digital value on a port.
- Setting the value of a bit as '0' in PxOUT register sets the output value of bit as low.
- Setting the value of a bit as '1' in PxOUT register sets the output value of bit as high.

For setting the value of a GPIO, set the pin as output using PxDIR register then set the value of bit in PxOUT register.



PxREN

- This eight bit register enables the pullup or pulldown resistor for a given pin.
- Setting the value of a bit as '0' in PxREN register disables the pullup/pulldown resistor .
- Setting the value of a bit as '1' in PxREN register enables the pullup/pulldown resistor.
- Once PxREN is set, PxOUT value is set to select between pullup or pulldown on pin. If PxOUT is set as '0' then pin is pulled down. If PxOUT is set as '1' then pin is pulled up.

PxSEL and PxSEL2

- Multiple functions are available on a single pin, PxSEL and PxSEL2 register are used to select function of a given pin.

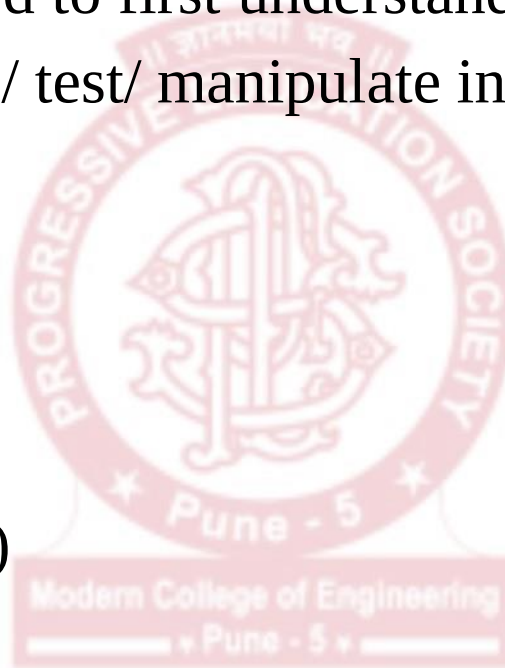
PxSEL2	PxSEL	Pin Function
0	0	I/O function is selected.
0	1	Primary peripheral module function is selected.
1	0	Reserved. See device-specific data sheet.
1	1	Secondary peripheral module function is selected.

Bit Manipulation

- In order to be able to configure the I/O and program the MSP430 we need to first understand some techniques to read/ assess/ configure/ test/ manipulate individual bits in a register.

Bitwise Operation:-

1. OR (|)
2. AND (&)
3. XOR (^)
4. Shift Right (>>)
5. Shift Left (<<)
6. NOT (~)



We will learn some of these operations with examples in the following slides.

Setting outputs

The I/Os are independently configurable!

Example: For Port P1, suppose there are 8 LEDs connected on each pin. Suppose we have to turn on the LED on P1.3, without changing any configuration on the rest of the port. How will we do it?

Based on the logic that $x \mid 0 = x$ and $x \mid 1 = 1$, there are a number of ways to set a bit:

1. `P1OUT = P1OUT | BIT3;`
2. `P1OUT |= BIT3;`
3. `P1OUT = P1OUT | 0x08;`

Setting outputs

Now we have to turn off the LED on P1.3, without changing the output on the rest of the port. How will we do it?

Based on the logic that $x \& 0 = 0$ and $x \& 1 = x$, there are a number of ways to clear a bit:

1. `P1OUT = P1OUT & (~BIT3);`
2. `P1OUT &= ~BIT3;`
3. `P1OUT = P1OUT & 0xF7;`

Similarly for toggling the output on any pin:

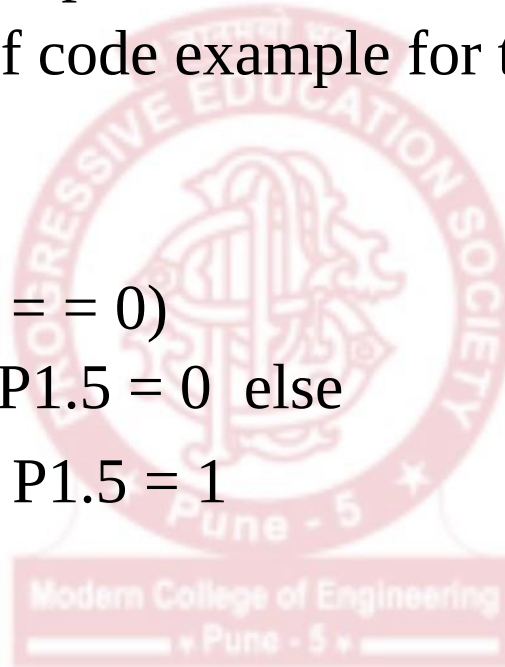
`P1OUT = P1OUT ^ BIT3;`

Reading inputs

Suppose there is a switch connected to P1.5 and we need to read the value of the pin.

Here is a snippet of code example for testing individual bits in a register:

```
if ((P1IN & BIT5) == 0)
//to be done when P1.5 = 0 else
// to be done when P1.5 = 1
```



C Program

Program in C to light LEDs with a constant pattern. Set P2.3 off and P2.4 on.

// Sets pins to output , lights pattern of LEDs , then loops forever

```
#include <msp430x11x1.h> // Specific device

void main ( void )
{
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    P2DIR = 0x18; // Set pins with LEDs to output , 0 b00011000
    P2OUT = 0x08; // LED2 (P2.4) on, LED1 (P2.3) off (active low!)
    for (;;) { // Loop forever...
        } // ...doing nothing
}
```

C Program

Program in C to light LED1 when button B1 is pressed. Both are on Port2

```
#include <msp430x11x1.h> // Specific device
#define LED1 BIT3
#define B1 BIT1
void main ( void )
{
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    P2OUT |= LED1; // Preload LED1 off (active low!)
    P2DIR = LED1; // Set pin with LED1 to output
    for (;;) { // Loop forever
        if ((P2IN & B1) == 0) // Is button down? (active low) {
            P2OUT &=~LED1; // Yes: Turn LED1 on (active low!)
        } else {
            P2OUT |= LED1; // No: Turn LED1 off (active low!)
        }
    }
}
```

C Program

Program in C to light LED1 when button B1 is pressed. Both are on Port2. This program has a loop for each state of the button.

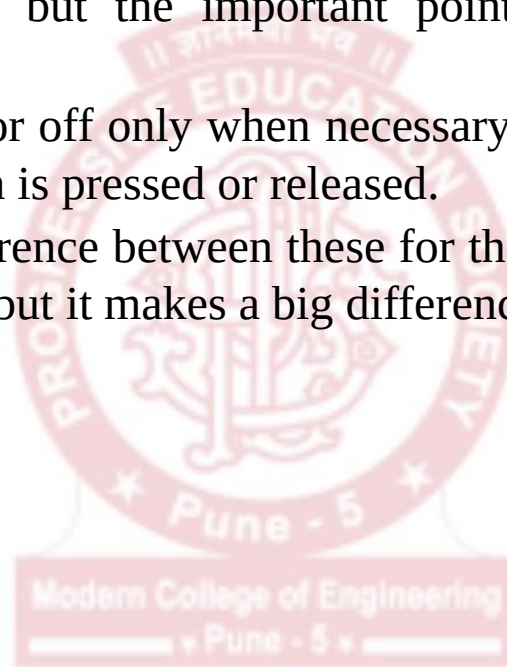
```
#include <msp430x11x1.h> // Specific device
#define LED1 BIT3
#define B1 BIT1
void main ( void )
{
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    P2OUT = LED1; // Preload LED1 off (active low!)
    P2DIR = LED1; // Set pin with LED1 to output
    for (;;) { // Loop forever
        while ((P2IN & B1) != 0) { // Loop while button up
            } // (active low) doing nothing
        // Actions to be taken when button is pressed
        P2OUT &= ~ LED1; // Turn LED1 on ( active low!)
        while ((P2IN & B1) == 0) { // Loop while button down
            } // (active low) doing nothing
        // Actions to be taken when button is released
        P2OUT |= LED1; // Turn LED1 off ( active low!)
    }
}
```

C Program

What is the difference between these two approaches?

1. The LED is continually being switched on or off in the first version. Of course this has no visible effect but the important point is that the action is repeated continually.
2. The LED is turned on or off only when necessary in the second version, just at the points when the button is pressed or released.

There is no practical difference between these for the simple task of lighting the LED while the button is down, but it makes a big difference for the following examples



C Program

Program in C to light LED1 when button B1 is pressed. Both are on Port2. This program uses bit fields rather than masks.

```
#include <io430x11x1.h> // Specific device , new format header
// Pins for LED and button
#define LED1 P2OUT_bit.P2OUT_3
#define B1 P2IN_bit.P2IN_1
void main ( void )
{
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    LED1 = 1; // Preload LED1 off (active low!)
    P2DIR_bit.P2DIR_3 = 1; // Set pin with LED1 to output
    for (;;) { // Loop forever
        while (B1 != 0) { // Loop while button up
            // (active low) doing nothing
        }
        // actions to be taken when button is pressed
        LED1 = 0; // Turn LED1 on ( active low!)
        while (B1 == 0) { // Loop while button down
            // (active low) doing nothing
        }
        // actions to be taken when button is released
        LED1 = 1; // Turn LED1 off (active low!)
    }
}
```

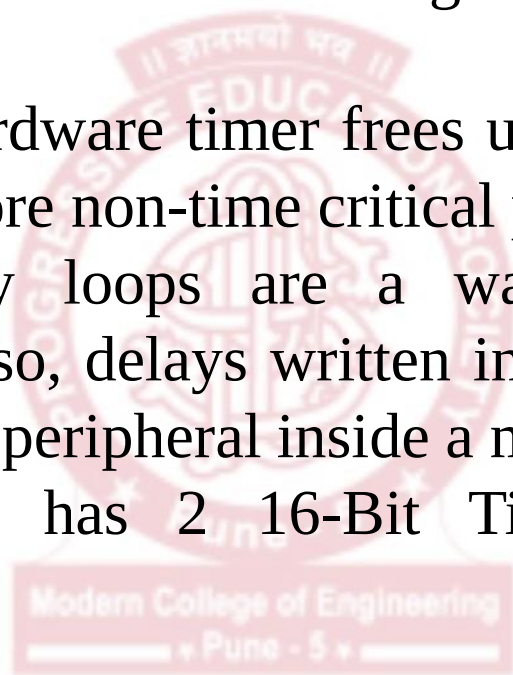
C Program

Program in C to light to to flash LEDs with a frequency of roughly 1Hz using a software delay. Toggle LEDs P2.3 and P2.4 with period of about 1s

```
#include <msp430x11x1.h> // Specific device
#define LED1 BIT3
#define LED2 BIT4
// Iterations of delay loop; reduce for simulation
#define DELAYLOOPS 50000
void main ( void )
{
    volatile unsigned int LoopCtr; // Loop counter: volatile!
    WDTCTL = WDTPW | WDTHOLD; // Stop watchdog timer
    P2OUT = ~ LED1; // Preload LED1 on, LED2 off
    P2DIR = LED1|LED2; // Set pins with LED1 ,2 to output
    for (;;) { // Loop forever
        for (LoopCtr = 0; LoopCtr < DELAYLOOPS; ++LoopCtr) {
            // Empty delay loop
            P2OUT ^= LED1|LED2; // Toggle LEDs
        }
    }
}
```

Why Timer?

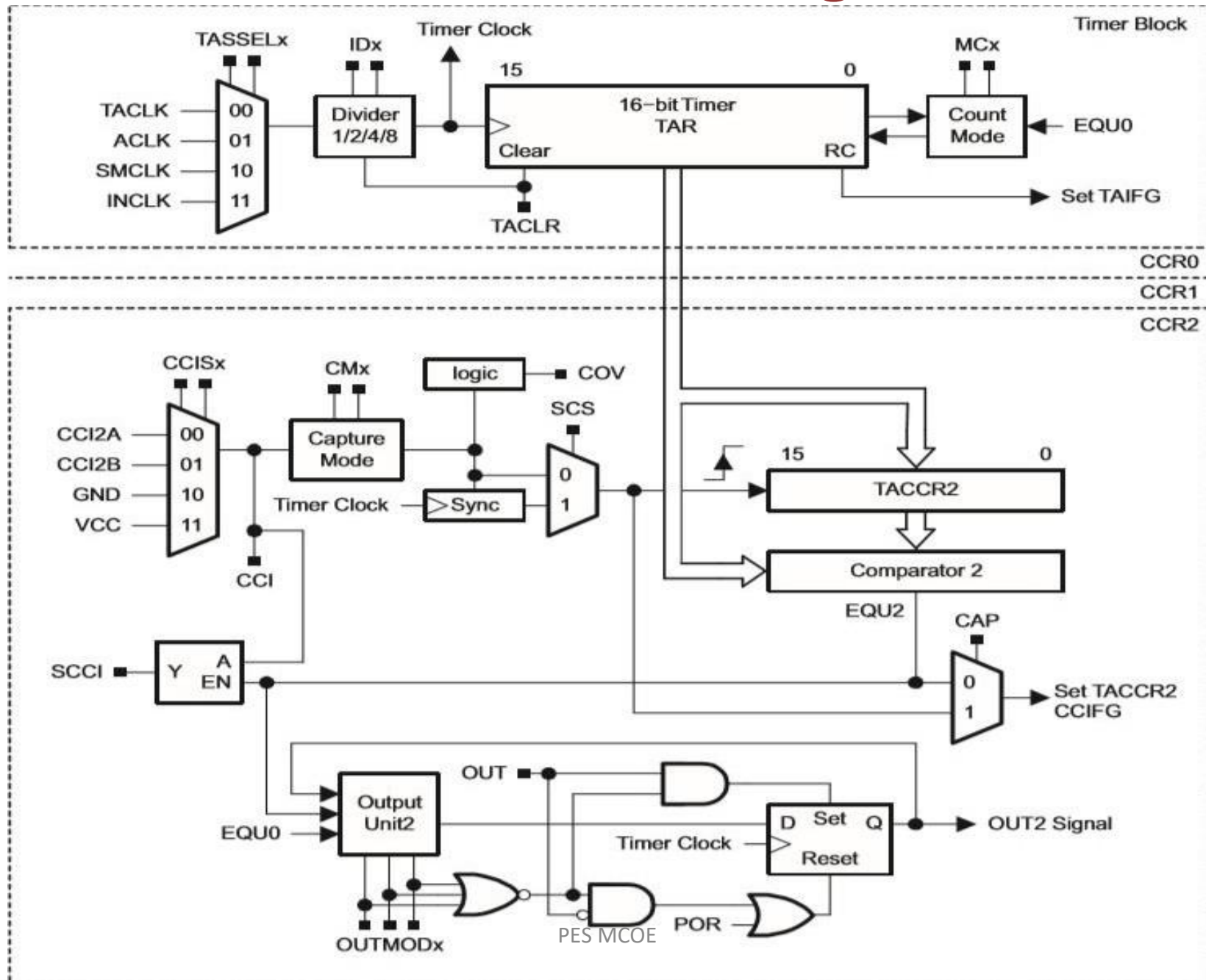
- Timers are used for time-specific tasks like generating delays, baud rates or for counting events in fixed intervals of time.
- A dedicated hardware timer frees up the processor so that it can perform more non-time critical process.
- Software delay loops are a waste of the processor's capabilities. Also, delays written in C are not precise. So, a hardware timer peripheral inside a microcontroller helps!
- MSP430G2553 has 2 16-Bit Timer_A: Timer0_A and Timer1_A.



Features of TIMER_A In MSP430G2553

- Timer_A is a 16-bit counter with four operating modes. It can count till 65535 ($2^{16}-1$).
- Timer_A can be used for capture/compare and PWM signals.
- It can also be used to generate interrupts, which may be generated from the counter on overflow conditions or from any of the capture/compare registers.
- It can measure frequency or take time-stamp of the external inputs directly.
- Outputs can be driven at specified times precisely, either once or periodically.

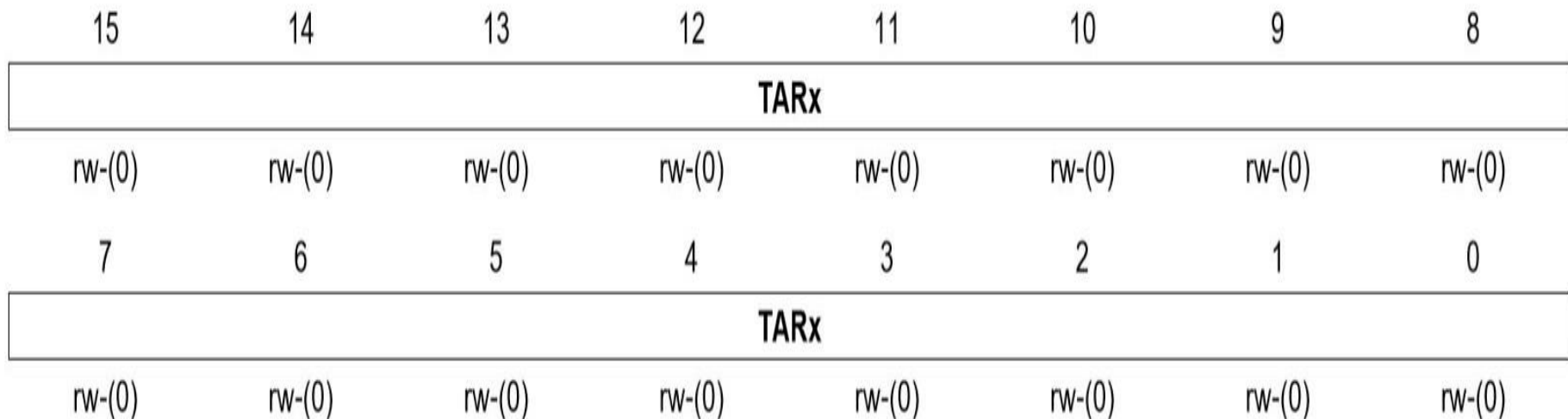
TIMER_A Block Diagram



TIMER_A Major Components

- **Timer_A counter register - TAR**

- This is a 16-bit register whose value increments or decrements with every rising edge of clock signal.
- Behaviour (increment or decrement) of TAR register can be selected by configuring the mode of the timer.



TARx

Bits 15-0 Timer_A register. The TAR register is the count of Timer_A.

TIMER_A Major Components

- **Timer_A capture/compare register (TACCRx)**
 - There are three capture/compare registers in each timer:
 - TA0CCR0, TA0CCR1, TA0CCR2 for Timer0_A
 - TA1CCR0, TA1CCR1, TA1CCR2 for Timer1_A.
 - All the three channels share the same TAR.

15	14	13	12	11	10	9	8
TACCRx							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
TACCRx							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

TACCRx

Bits 15-0

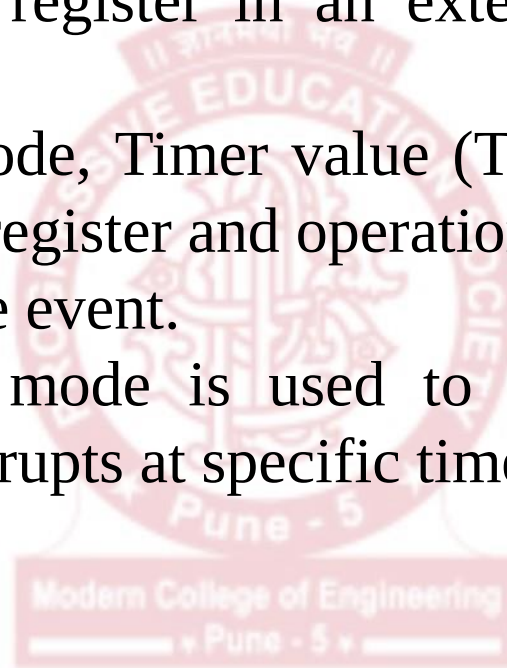
Timer_A capture/compare register.

Compare mode: TACCRx holds the data for the comparison to the timer value in the Timer_A Register, TAR.

Capture mode: The Timer_A Register, TAR, is copied into the TACCRx register when a capture is performed.

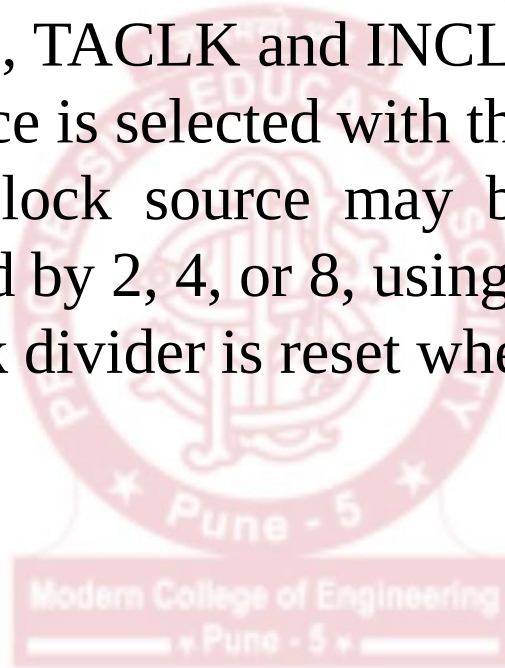
Capture Vs Compare Mode

- In capture mode, the Timer value (TAR) is captured in this TACCRx register in an external or internal input event.
- In compare mode, Timer value (TAR) is compared with this TACCRx register and operations are performed based on the compare event.
- The compare mode is used to generate PWM output signals or interrupts at specific time intervals.



Clock Source for Timer_A

- The timer clock can be sourced from ACLK, SMCLK, or externally via TACLK or INCLK (For Timer1_A in case of MSP430G2553, TACLK and INCLK is not present).
- The clock source is selected with the TASSELx bits.
- The selected clock source may be passed directly to the timer or divided by 2, 4, or 8, using the IDx bits.
- The timer clock divider is reset when TACLR is set.



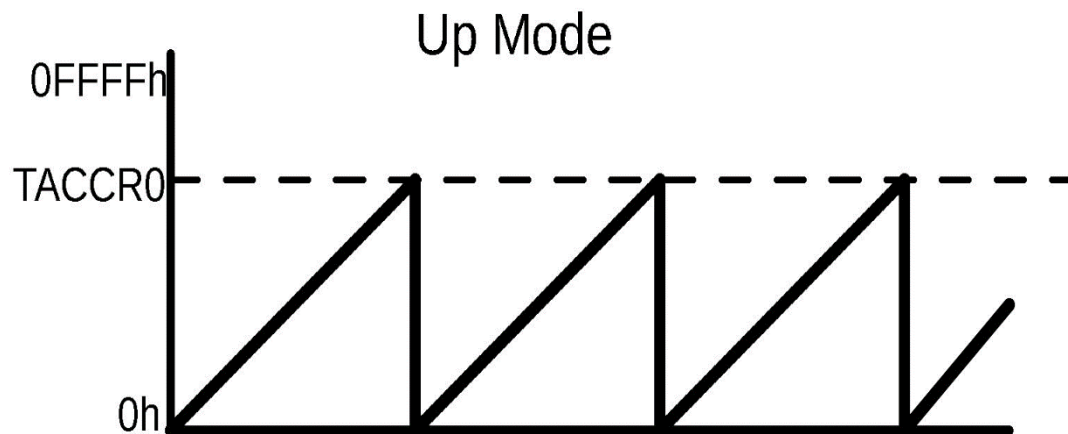
Timer Modes

Timer's operation mode can be selected by configuring 'MCx' bits in the Timer Control TACTL register

MC	Mode	Description
00	Stop	Timer is stopped.
01	Up	Timer repeatedly counts from zero to value stored in TACCR0 Register
10	Continuous	Timer repeatedly counts from zero to 0xFFFF hex.
11	Up/Down	Timer repeatedly counts from zero to value stored in TACCR0 and then back to zero.

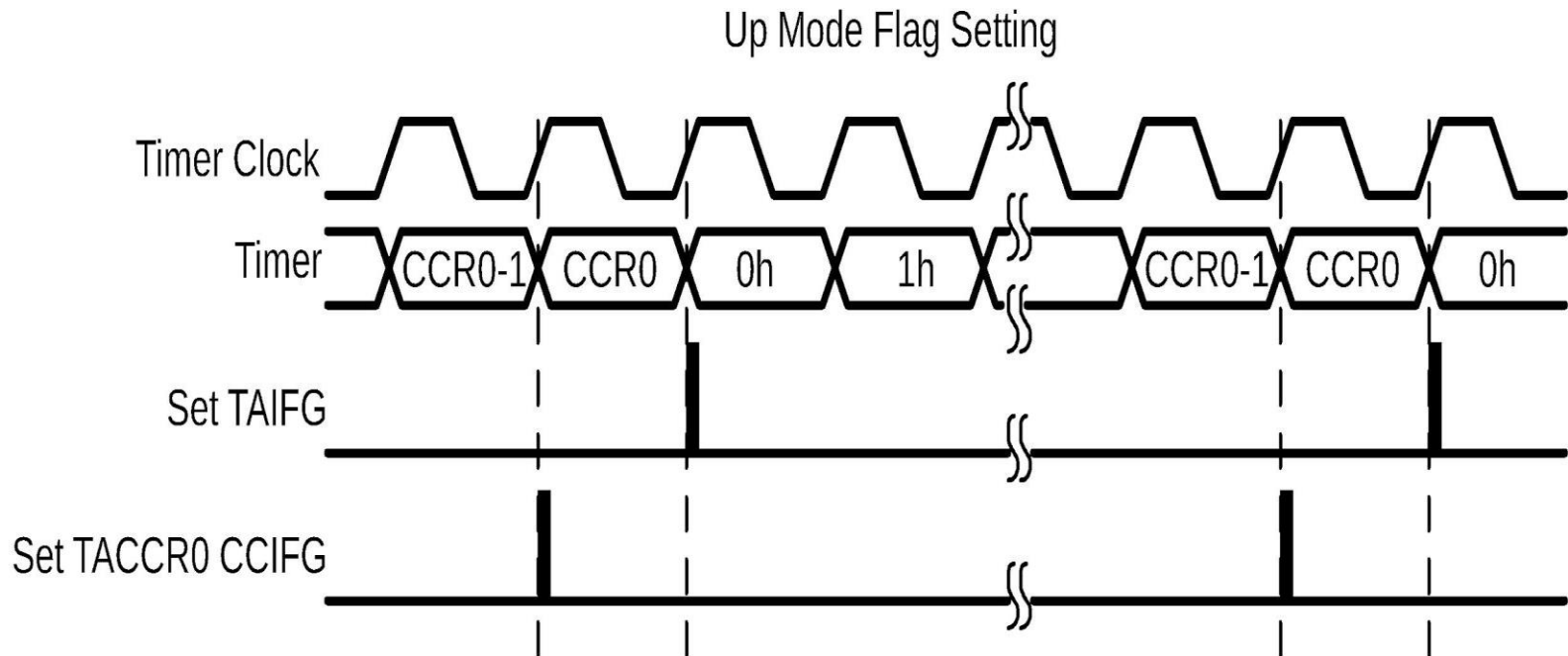
Up Mode

- The timer repeatedly counts up to the value of compare register TACCR0 starting from 0000h.
- The number of timer counts in the period is $TACCR0+1$. When the timer value equals TACCR0 the timer restarts counting from zero.
- The up mode is used if the timer period is required to be different from 0FFFFh counts.



Timer Interrupts in Up Mode

- The TACCR0 CCIFG interrupt flag is set when the timer *counts* to the TACCR0 value.
- The TAIFG interrupt flag is set when the timer *counts* from TACCR0 to zero.

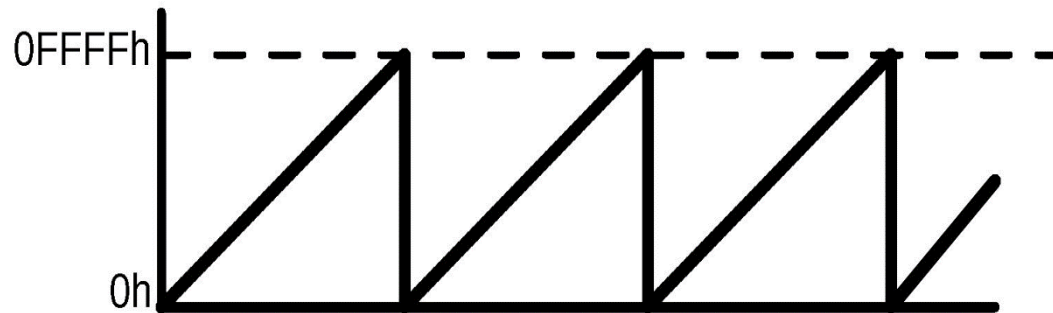


Continuous Mode

- In the continuous mode, the timer repeatedly counts up to FFFFh and restarts from zero.



Continuous Mode

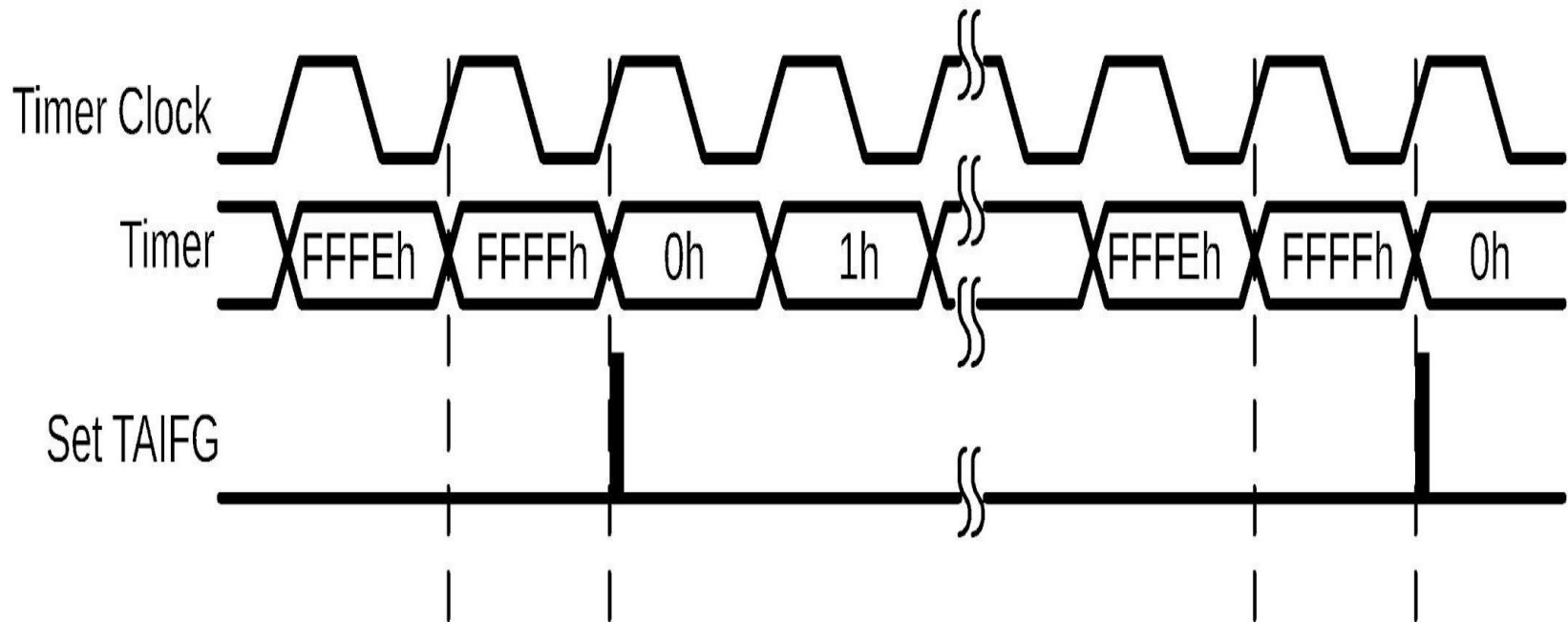


Timer Interrupts in Continuous Mode

- The TAIIFG interrupt flag is set when the timer *counts* from FFFFh to zero.

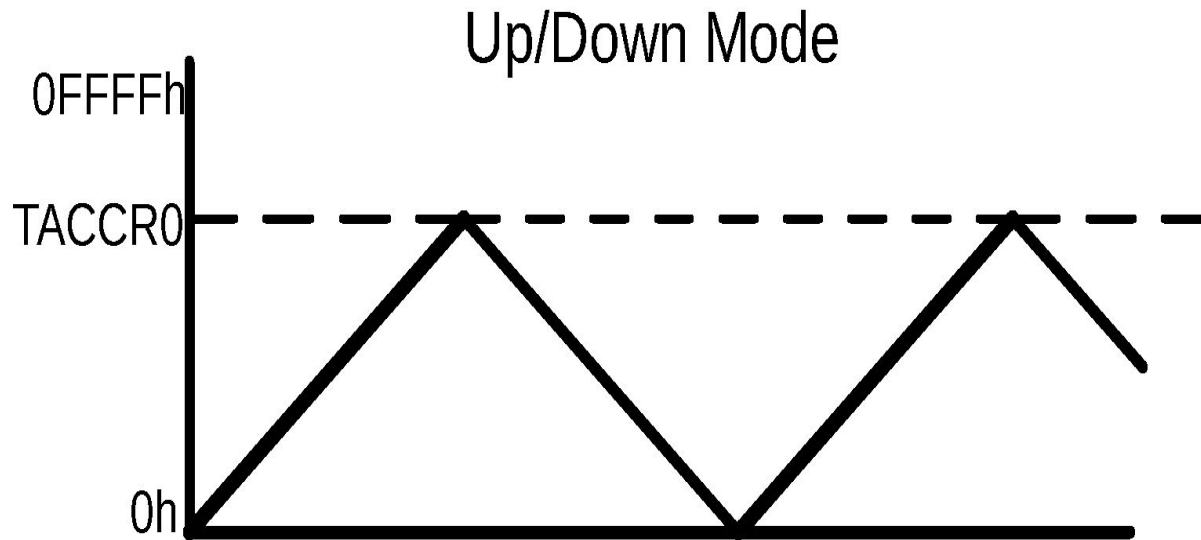


Continuous Mode Flag Setting



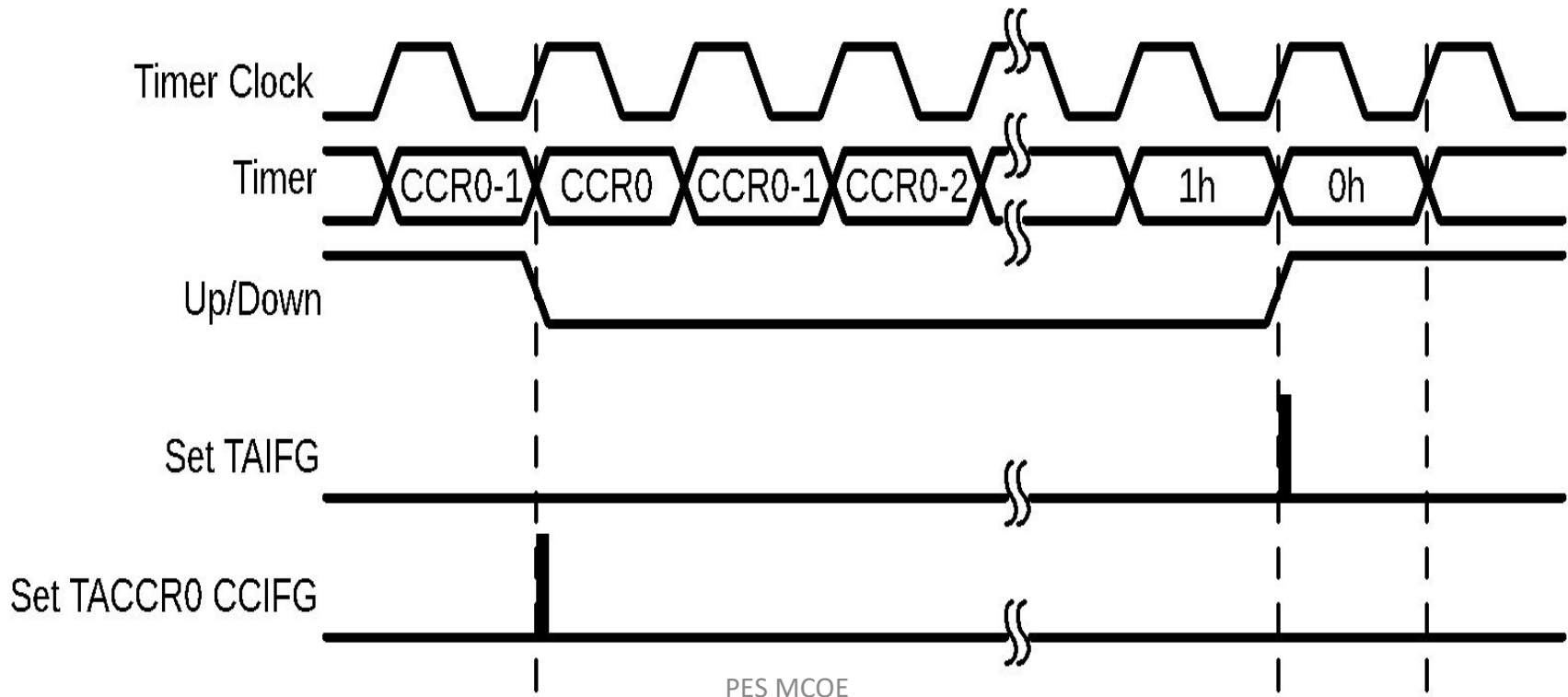
Up/Down Mode

- The timer repeatedly counts up to the value of compare register TACCR0 and back down to zero.
- The up/down mode is used if the timer period must be different from 0FFFFh counts, and if a symmetrical pulse generation is needed.



Timer Interrupts in Up/Down Mode

- The TACCR0 CCIFG interrupt flag is set when the timer counts from TACCR0 – 1 to TACCR0
- TAIFG is set when the timer completes counting down from 0001h to 0000h.



TIMER_A Registers

1. Timer_A control - TACTL
2. Timer_A counter - TAR
3. Timer_A capture/compare control 0 - TACCTL0
4. Timer_A capture/compare 0 - TACCR0
5. Timer_A capture/compare control 1 - TACCTL1
6. Timer_A capture/compare 1 - TACCR1
7. Timer_A capture/compare control 2 - TACCTL2
8. Timer_A capture/compare 2 - TACCR2
9. Timer_A Interrupt vector - TAIIV

TACTL, Timer_A Control Register

15	14	13	12	11	10	9	8
Unused						TASSELx	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
IDx		MCx		Unused	TACLR	TAIE	TAIFG
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

TACTL Continued

TASSELx	Bits 9-8	Timer_A clock source select
		00 TACLK
		01 ACLK
		10 SMCLK
		11 INCLK (INCLK is device-specific and is often assigned to the inverted TBCLK) (see the device-specific data sheet)
IDx	Bits 7-6	Input divider. These bits select the divider for the input clock.
		00 /1
		01 /2
		10 /4
		11 /8
MCx	Bits 5-4	Mode control. Setting MCx = 00h when Timer_A is not in use conserves power.
		00 Stop mode: the timer is halted.
		01 Up mode: the timer counts up to TACCR0.
		10 Continuous mode: the timer counts up to 0FFFFh.
		11 Up/down mode: the timer counts up to TACCR0 then down to 0000h.
Unused	Bit 3	Unused
TACLR	Bit 2	Timer_A clear. Setting this bit resets TAR, the clock divider, and the count direction. The TACLR bit is automatically reset and is always read as zero.
TAIE	Bit 1	Timer_A interrupt enable. This bit enables the TAIFG interrupt request.
		0 Interrupt disabled
		1 Interrupt enabled
TAIFG	Bit 0	Timer_A interrupt flag
		0 No interrupt pending
		1 Interrupt pending

TACCTLx, Capture/Compare Control Register

15	14	13	12	11	10	9	8
CMx		CCISx		SCS	SCCI	Unused	CAP
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	r	r0	rw-(0)
7	6	5	4	3	2	1	0
OUTMODx			CCIE	CCI	OUT	COV	CCIFG
rw-(0)	rw-(0)	rw-(0)	rw-(0)	r	rw-(0)	rw-(0)	rw-(0)

Timer_A has 3 channels each being controlled by TACCTLx.
Each channel has its TACCRx register.

TACCTLx Continued

CMx	Bit 15-14	Capture mode	
		00	No capture
		01	Capture on rising edge
		10	Capture on falling edge
		11	Capture on both rising and falling edges
CCISx	Bit 13-12	Capture/compare input select. These bits select the TACCRx input signal. See the device-specific data sheet for specific signal connections.	
		00	CClxA
		01	CClxB
		10	GND
		11	V _{CC}
SCS	Bit 11	Synchronize capture source. This bit is used to synchronize the capture input signal with the timer clock.	
		0	Asynchronous capture
		1	Synchronous capture
SCCI	Bit 10	Synchronized capture/compare input. The selected CCI input signal is latched with the EQUx signal and can be read via this bit	
Unused	Bit 9	Unused. Read only. Always read as 0.	
CAP	Bit 8	Capture mode	
		0	Compare mode
		1	Capture mode

TACCTLx Continued

OUTMODx	Bits 7-5	Output mode. Modes 2, 3, 6, and 7 are not useful for TACCR0, because EQUx = EQU0.
		000 OUT bit value
		001 Set
		010 Toggle/reset
		011 Set/reset
		100 Toggle
		101 Reset
		110 Toggle/set
		111 Reset/set
CCIE	Bit 4	Capture/compare interrupt enable. This bit enables the interrupt request of the corresponding CCIFG flag.
		0 Interrupt disabled
		1 Interrupt enabled
CCI	Bit 3	Capture/compare input. The selected input signal can be read by this bit.
OUT	Bit 2	Output. For output mode 0, this bit directly controls the state of the output.
		0 Output low
		1 Output high
COV	Bit 1	Capture overflow. This bit indicates a capture overflow occurred. COV must be reset with software.
		0 No capture overflow occurred
		1 Capture overflow occurred
CCIFG	Bit 0	Capture/compare interrupt flag
		0 No interrupt pending
		1 Interrupt pending

TAIV, Timer_A Interrupt Vector Register

15	14	13	12	11	10	9	8
0	0	0	0	0	0	0	0
r0	r0	r0	r0	r0	r0	r0	r0
7	6	5	4	3	2	1	0
0	0	0	0	TAIVx			0
r0	r0	r0	r0	r-(0)	r-(0)	r-(0)	r0

TAIVx Bits 15-0 Timer_A interrupt vector value

TAIV Contents	Interrupt Source	Interrupt Flag	Interrupt Priority
00h	No interrupt pending	-	
02h	Capture/compare 1	TACCR1 CCIFG	Highest
04h	Capture/compare 2 ⁽¹⁾	TACCR2 CCIFG	
06h	Reserved	-	
08h	Reserved	-	
0Ah	Timer overflow	TAIFG	
0Ch	Reserved	-	
0Eh	Reserved	-	Lowest

⁽¹⁾ Not implemented in MSP430x20xx devices

TIMER_A Interrupts

- Timer Overflow sets the flag TAIIFG in TACTL.
- Interrupt vectors associated with the 16-bit Timer_A module:
 - TACCR0 interrupt vector for TACCR0 CCIFG.
 - TAIIV interrupt vector for all other CCIFG flags and TAIIFG.
- In capture mode any CCIFG flag is set when a timer value is captured in the associated TACCRx register.
- In compare mode, any CCIFG flag is set if TAR counts to the associated TACCRx value.
- Software may also set or clear any CCIFG flag. All CCIFG flags request an interrupt when their corresponding CCIE bit and the GIE bit are set.

Capture Mode

- The capture mode retrieves a timer value based on a signal event.
- It can be used to measure the amount of time between signals.
- A single event could be represented by either the rising or falling edge of a signal.
- When a timer works in capture mode, it will be monitoring an input signal and waiting for that signal event to occur. When a selected edge is detected, the time of the event occurrence will be recorded by latching the current timer count.
- Using an MCU timer in this input capture mode, we could measure the width of a signal or the period of a signal easily, based on different signal edge configurations.
- This feature is usually used with IR captures, encoder inputs, and communication protocol applications.

Capture Mode

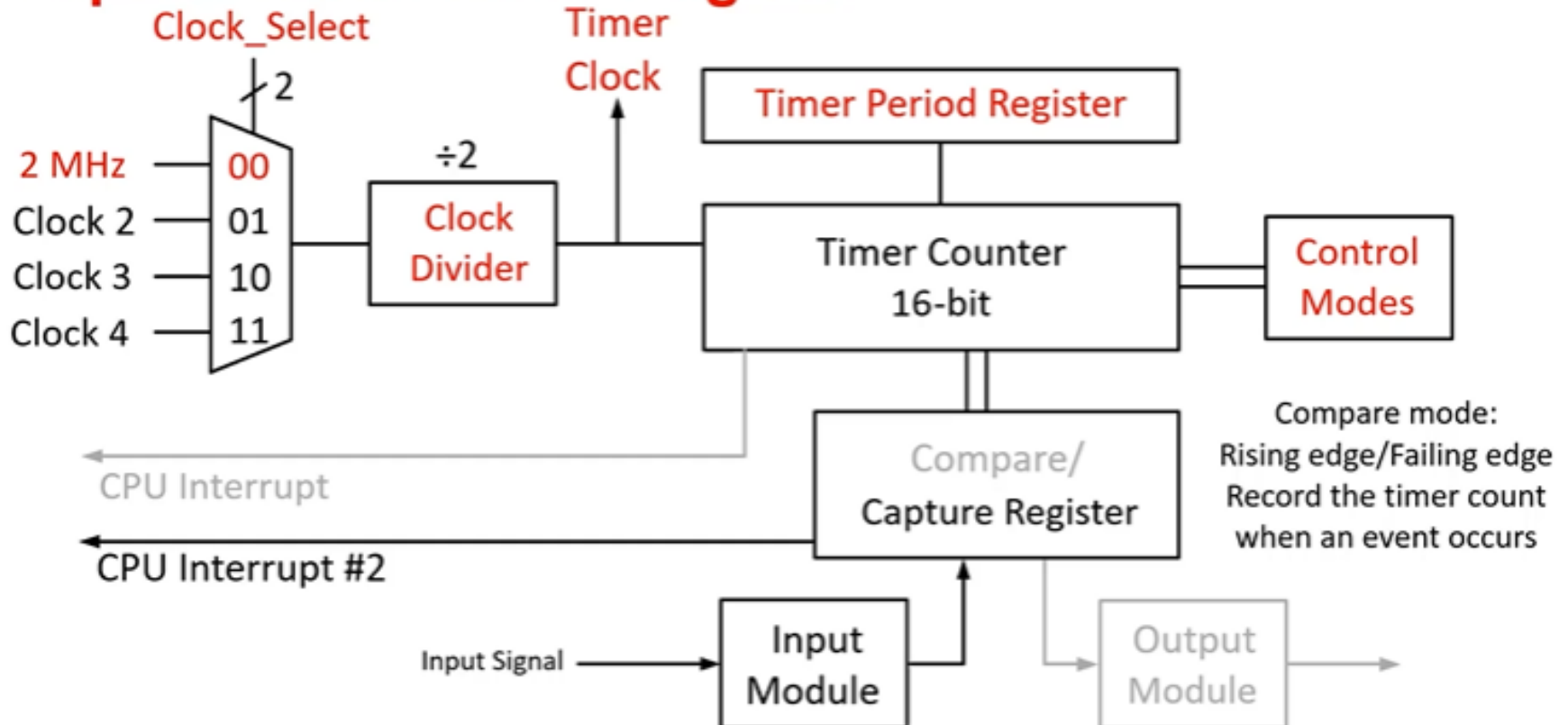
Capture Mode - retrieves a timer value based on a **signal event**.

Signal Event - can be represented by the rising and falling edges of a pulse.

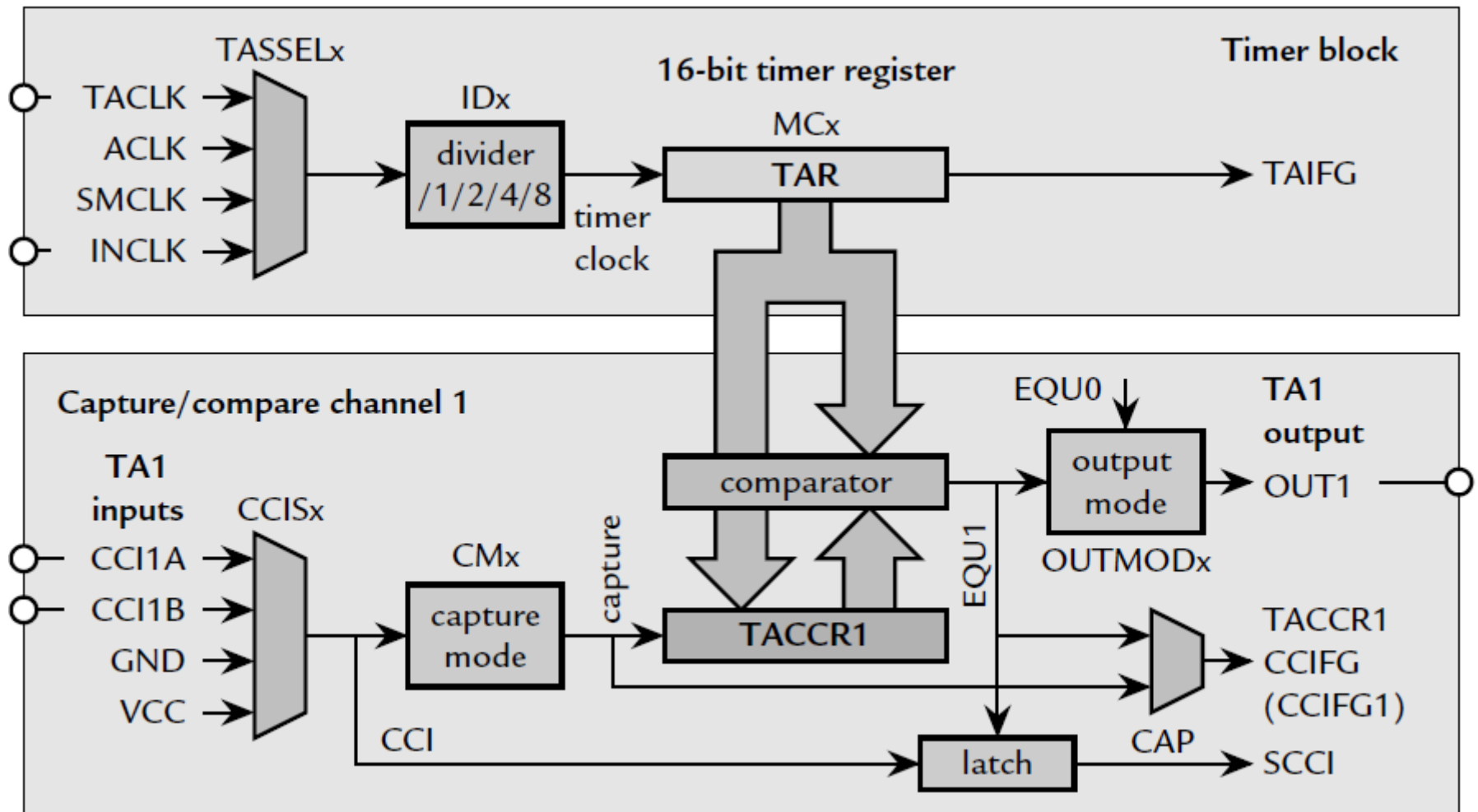


Capture Mode

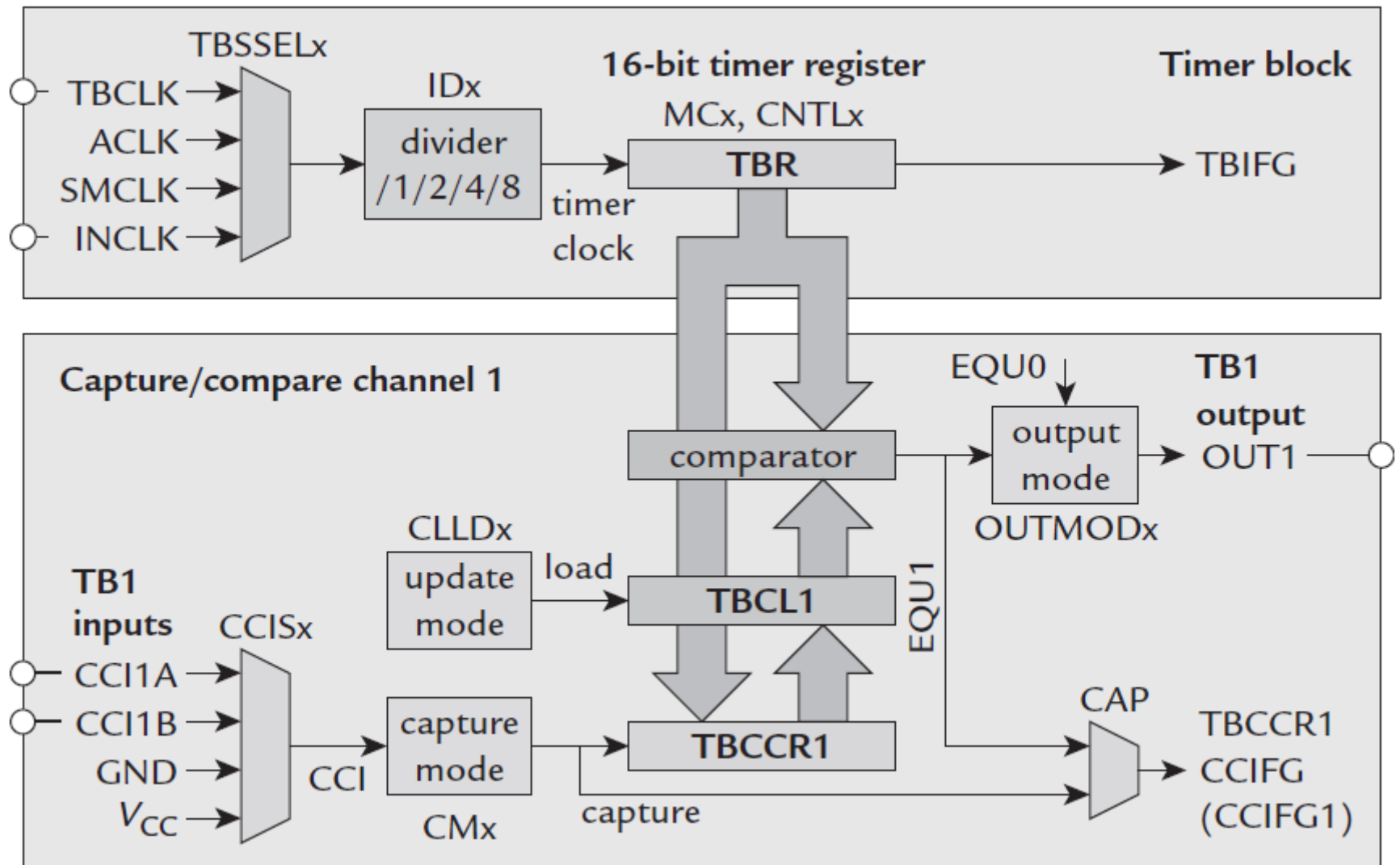
Capture mode block diagram



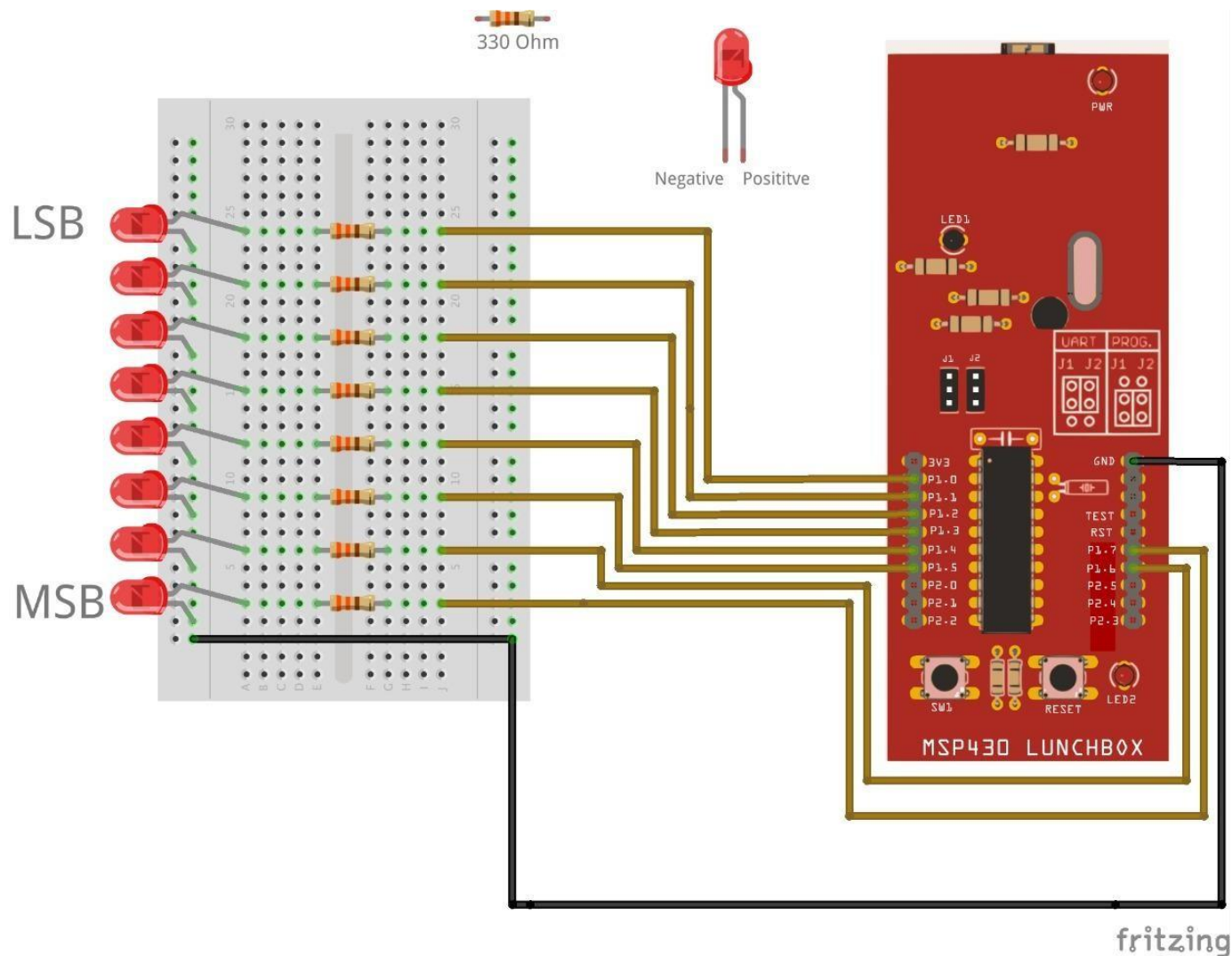
Capture Mode



Compare Mode



Circuit Diagram



PES MCOE

Example Code : Hello_Timer

```
1#include <msp430.h>
2#include <inttypes.h>
3
4volatile char count = 0;
5volatile unsigned int i;
6
7/**
8 * @brief
9 * These settings are wrt enabling GPIO on Lunchbox
10 */
11void register_settings_for_GPIO()
12{
13    P1DIR |= 0xFF;           //P1.0 to P1.7 are set as Output
14    P1OUT |= 0x00;          //Initially they are set to logic zero
15}
16
17/**
18 * @brief
19 * These settings are w.r.t enabling TIMER0 on Lunch Box
20 */
21void register_settings_for_TIMER0()
22{
23    CCTL0 = CCIE;           // CCR0 interrupt enabled
24    TACTL = TASSEL_1 + MC_1; // ACLK = 32768 Hz, upmode
25    CCR0 = 32768;           // 1 Hz
26}
```

Example Code : Hello_Timer

```
27
28 /*@brief entry point for the code*/
29 void main(void)
30 {
31     WDTCTL = WDTPW + WDTHOLD;          ///! Stop Watch dog (Not recommended for code in production and devices working in field)
32
33     do{
34         IFG1 &= ~OFIFG;                // Clear oscillator fault flag
35         for (i = 50000; i; i--);       // Delay
36     } while (IFG1 & OFIFG);            // Test osc fault flag
37
38     register_settings_for_TIMER0();
39     register_settings_for_GPIO();
40     _BIS_SR(LPM3_bits + GIE);          // Enter LPM3 w/ interrupt
41 }
42
43 /*@brief entry point for TIMER0 interrupt vector*/
44 #pragma vector= TIMER0_A0_VECTOR
45 __interrupt void Timer_A (void)
46 {
47     count++;
48     P1OUT = count;                     //Assign value of Count to PORT 1 to represent it as binary number
49 }
```

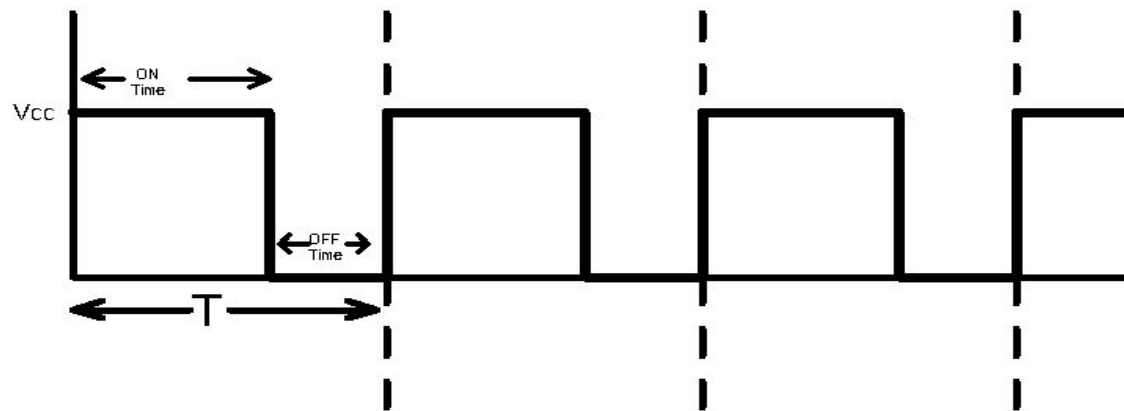
Pulse Width Modulation

Timer Compare: Pulse Width Modulation In MSP430



Pulse Width Modulation

- PWM is a technique in which the width of a pulse is modulated to vary the average value of the signal.
- The width refers to the ON-time of the Rectangular pulse. The time period of the signal is kept fixed.

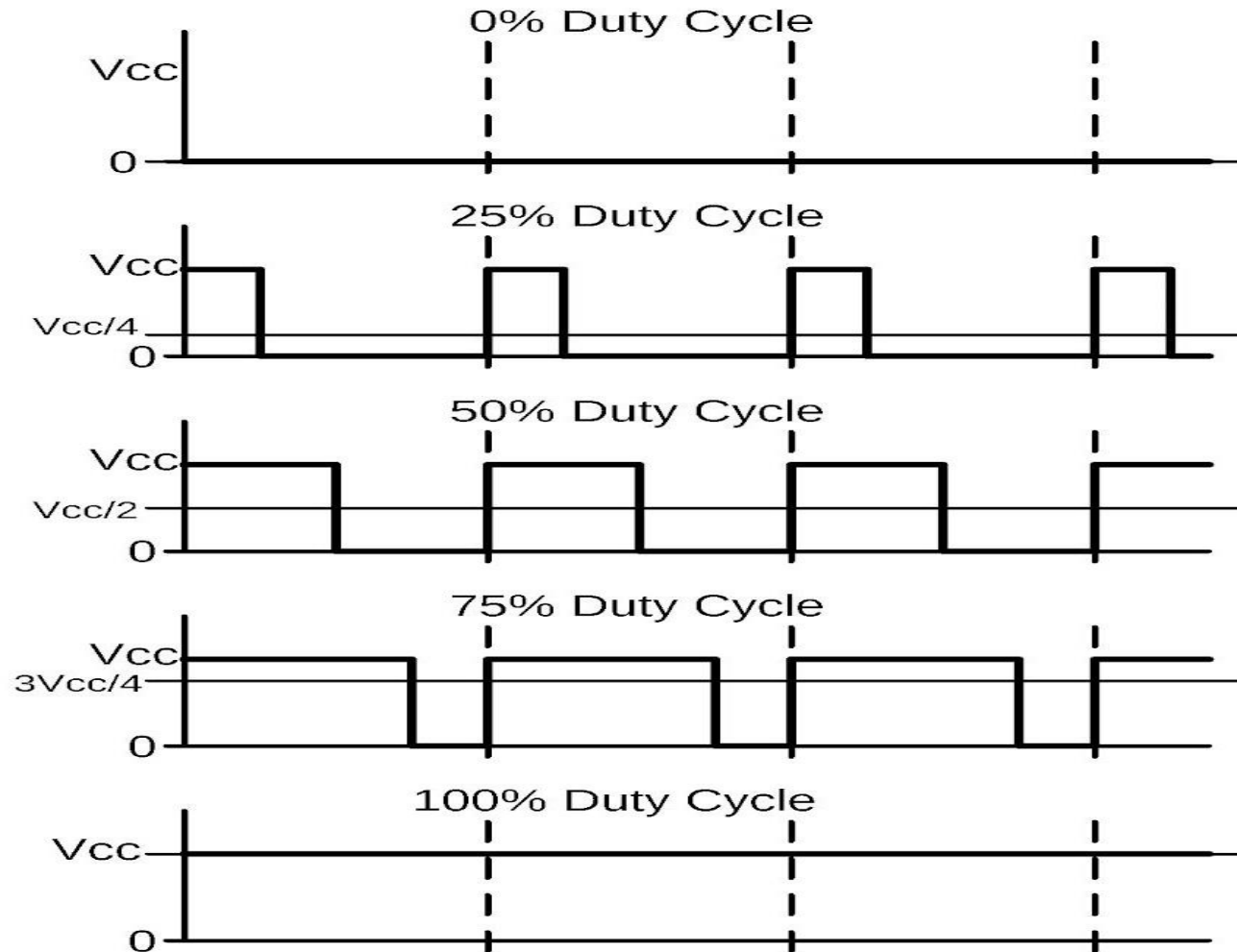


$$\text{Duty Cycle} = \frac{T_{\text{on}}}{T_{\text{on}} + T_{\text{off}}}$$

Pulse Width Modulation

- As the On-Time increases, the Duty Cycle of the Pulse increases and the average value of the signal increases.
- If a PWM signal is applied to an LED such that it is turned on and off at a very high rate that the human eye cannot detect the changes due to persistence of vision, the intensity of the led seems to vary.
- The output device operates on average value of this pulse.
- This mimics analog voltages through a digital output.

Average Value of Signals of Different Duty Cycles



Software PWM

- Software PWM is implemented with the help of delays.

Step 1: Turn the Output Pin On using software by using PxOUT

Step 2: Give some delay (Ton)

Step 3: Turn the Output Pin Off using software by using PxOUT

Step 4: Give some delay (Toff)

- Software PWM is helpful when the pin cannot be configured as a Timer Output Pin.
- PWM Time Period = $T_{on} + T_{off}$.

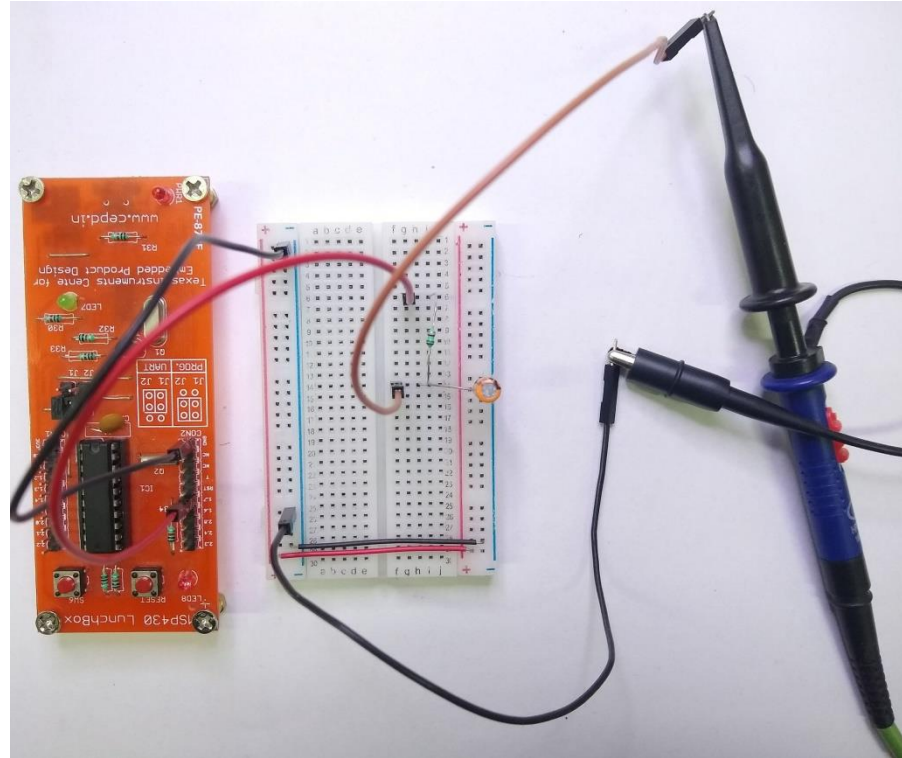
Software PWM Code with Large Delay

```
1 #include <msp430.h>
2
3 #define RED BIT7 // Red LED -> P1.7
4
5 /**
6  * @brief This function provides delay
7  * @param unsigned int
8  * @return void
9  */
10 void delay(unsigned int t) // Custom delay function
11 {
12     unsigned int i;
13     for(i = t; i > 0; i--)
14     {
15         __delay_cycles(50); // __delay_cycles accepts only constants !
16     }
17 }
18
19 /* @brief entry point for the code */
20 void main(void) {
21     unsigned int j;
22     WDCTL = WDTPW | WDTHOLD; // Stop watchdog timer
23
24     P1DIR |= RED; // Set LED pin -> Output
25
26     while(1)
27     {
28
29         for(j = 0; j < 256; j++) // Increasing Intensity
30         {
31             P1OUT |= RED; // LED ON
32             if (j != 0) delay(j); // Delay for ON Time
33             P1OUT &= ~RED; // LED OFF
34             if ((255-j) != 0) delay(255-j); // OFF Time = Period - ON Time
35         }
36         for(j = 255; j > 0; j--) // Decreasing Intensity
37         {
38             P1OUT |= RED; // LED ON
39             if (j != 0) delay(j); // Delay for ON Time
40             P1OUT &= ~RED; // LED OFF
41             if ((255-j) != 0) delay(255-j); // OFF Time = Period - ON Time
42         }
43     }
44 }
```

Software PWM Code with Less Delay

```
1 #include <msp430.h>
2
3 #define RED    BIT7                // Red LED -> P1.7
4
5 /**
6  * @brief This function provides delay
7  * @param unsigned int
8  * @return void
9  */
10 void delay(unsigned int t)          // Custom delay function
11 {
12     unsigned int i;
13     for(i = t; i > 0; i--)
14     {
15         __delay_cycles(1);          // __delay_cycles accepts only constants !
16     }
17 }
18
19 /*@brief entry point for the code*/
20 void main(void) {
21     unsigned int j;
22     WDTCTL = WDTPW | WDTHOLD;       // stop watchdog timer
23
24     P1DIR |= RED;                   // Set LED pin -> Output
25
26     while(1)
27     {
28         for(j = 0; j < 256; j++)    // Increasing Intensity
29         {
30             P1OUT |= RED;           // LED ON
31             if (j != 0) delay(j);   // Delay for ON Time
32             P1OUT &= ~RED;          // LED OFF
33             if ((255-j) != 0) delay(255-j); // OFF Time = Period - ON Time
34         }
35         for(j = 255; j > 0; j--)    // Decreasing Intensity
36         {
37             P1OUT |= RED;           // LED ON
38             if (j != 0) delay(j);   // Delay for ON Time
39             P1OUT &= ~RED;          // LED OFF
40             if ((255-j) != 0) delay(255-j); // OFF Time = Period - ON Time
41         }
42     }
43 }
44
```

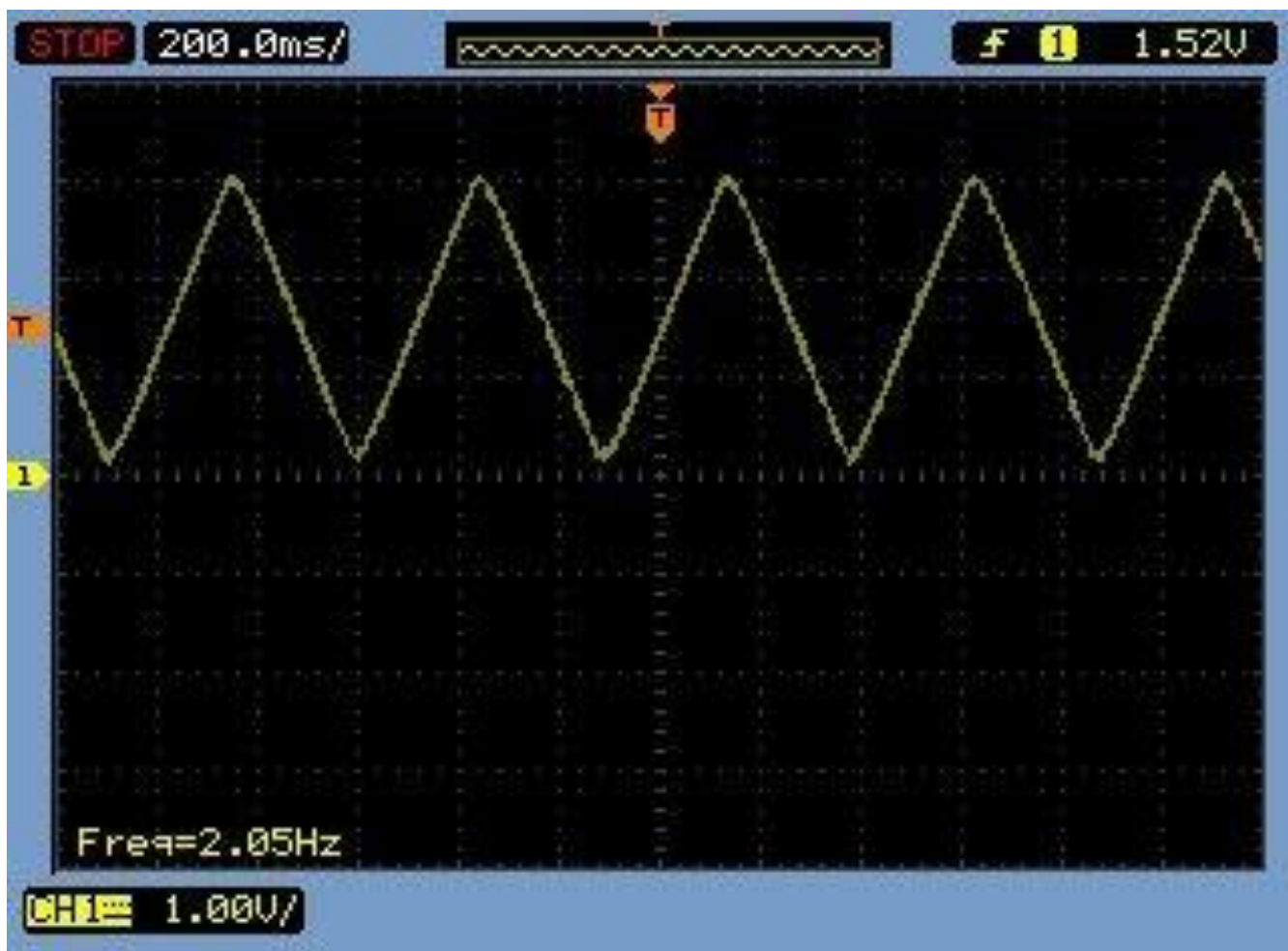
Filter Connection



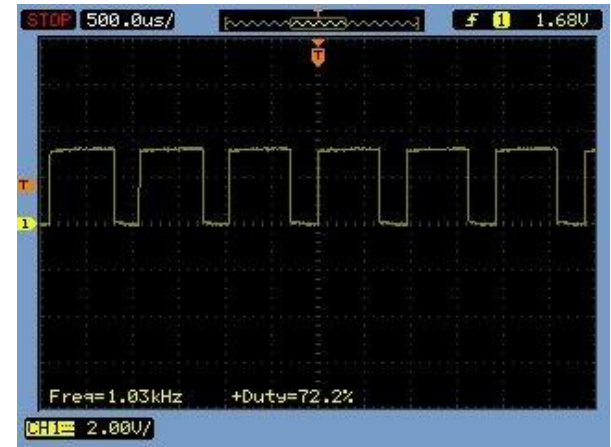
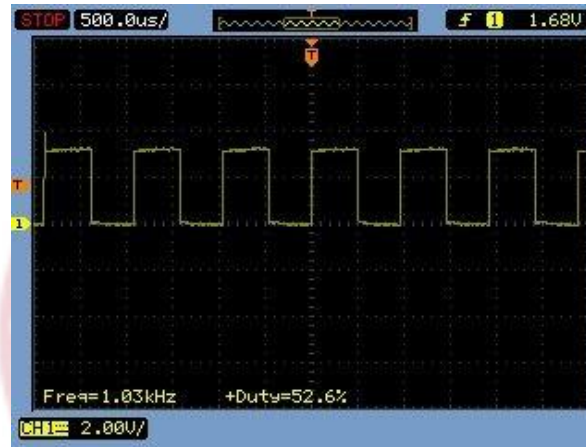
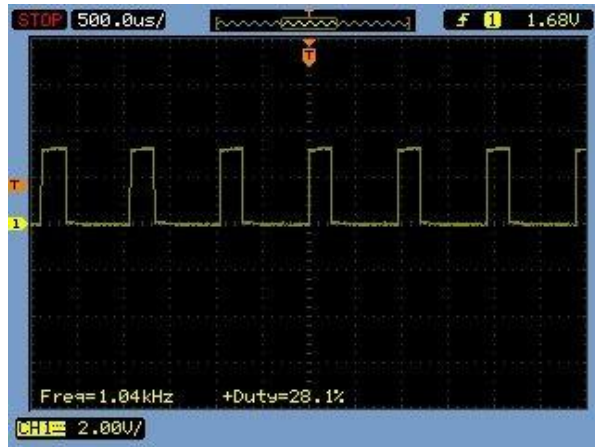
$R = 1.5 \text{ Kohm}, C = 10 \text{ uF}$

Filter Frequency = $1/(2 \cdot 3.14 \cdot R \cdot C) = 10.6 \text{ Hz}$

Filtered Output



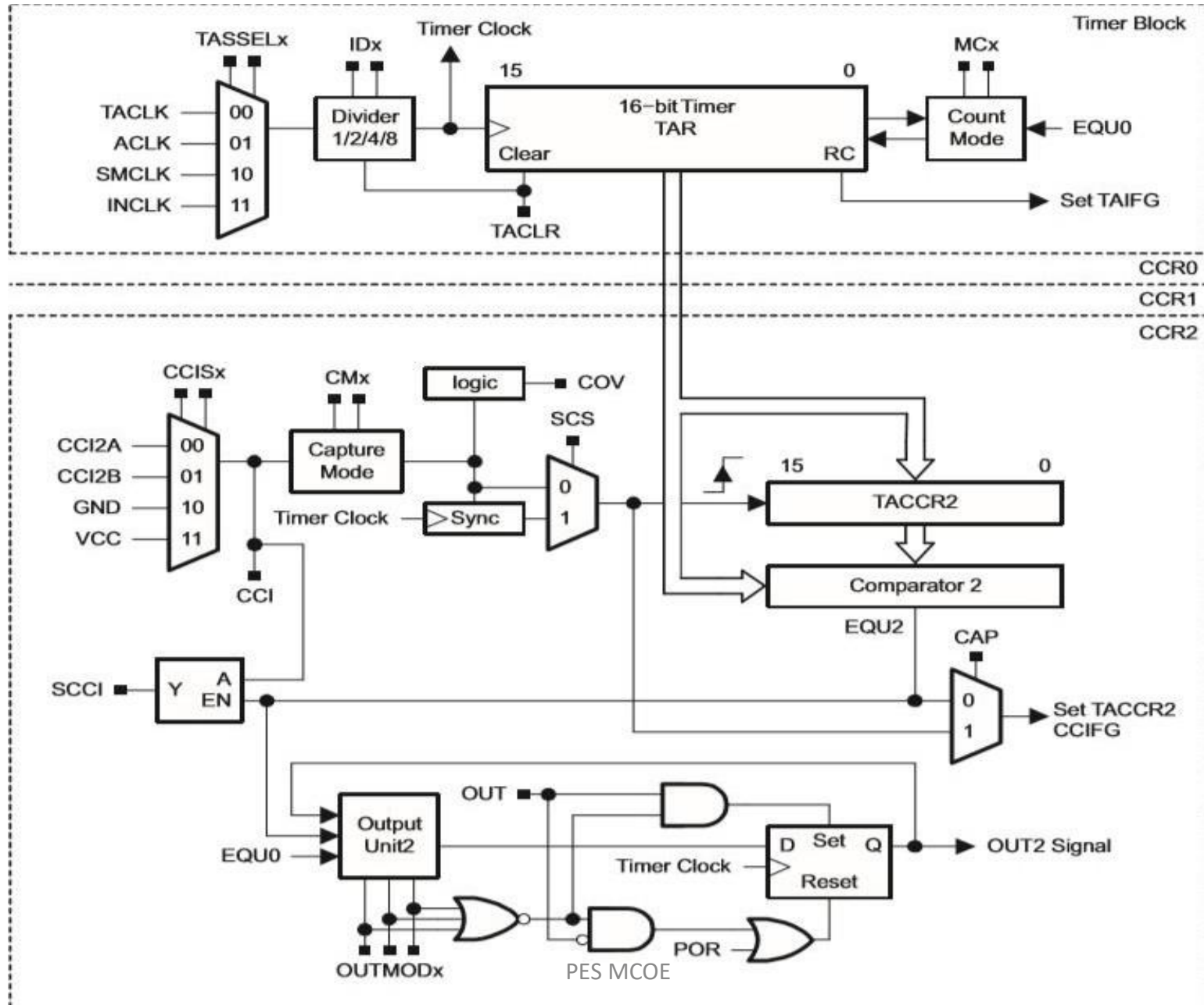
PWM signal at pin 1.7



Hardware PWM



TIMER_A Block Diagram



TIMER_A Output Unit

- Each capture/compare block contains an Output Unit.
- The compare mode is selected when bit CAP = 0 in TACCTLx register.
- When TAR value equals value in TACCRx, a internal signal EQUx is set, which affects the output according to output mode.
- The output unit is used to generate output signals such as PWM signals.
- Each output unit has eight operating modes that generate signals based on the EQU0 and EQUx signals.
- The output modes are defined by the OUTMODn bits in TACCTLx register

Timer Modes

Timer's operation mode can be selected by configuring 'MCx' bits in the Timer Control TACTL register

MC	Mode	Description
00	Stop	Timer is stopped.
01	Up	Timer repeatedly counts from zero to value stored in TACCR0 Register
10	Continuous	Timer repeatedly counts from zero to 0xFFFF hex.
11	Up/Down	Timer repeatedly counts from zero to value stored in TACCR0 and then back to zero.

TIMER_A Output Modes

OUTMOD	Mode	Description
000	Output	The output signal OUTn is defined by the OUT bit. The OUTn signal updates immediately when OUT is updated.
001	Set	The output is set when the timer <i>counts</i> to the TAxCCRn value. It remains set until a reset of the timer, or until another output mode is selected and affects the output.
010	Toggle/Reset	The output is toggled when the timer <i>counts</i> to the TAxCCRn value. It is reset when the timer <i>counts</i> to the TAxCCR0 value.
011	Set/Reset	The output is set when the timer <i>counts</i> to the TAxCCRn value. It is reset when the timer <i>counts</i> to the TAxCCR0 value.
100	Toggle	The output is toggled when the timer <i>counts</i> to the TAxCCRn value. The output period is double the timer period.
101	Reset	The output is reset when the timer <i>counts</i> to the TAxCCRn value. It remains reset until another output mode is selected and affects the output.
110	Toggle/Set	The output is toggled when the timer <i>counts</i> to the TAxCCRn value. It is set when the timer <i>counts</i> to the TAxCCR0 value.
111	Reset/Set	The output is reset when the timer <i>counts</i> to the TAxCCRn value. It is set when the timer <i>counts</i> to the TAxCCR0 value.

Output Mode Example with Up Mode

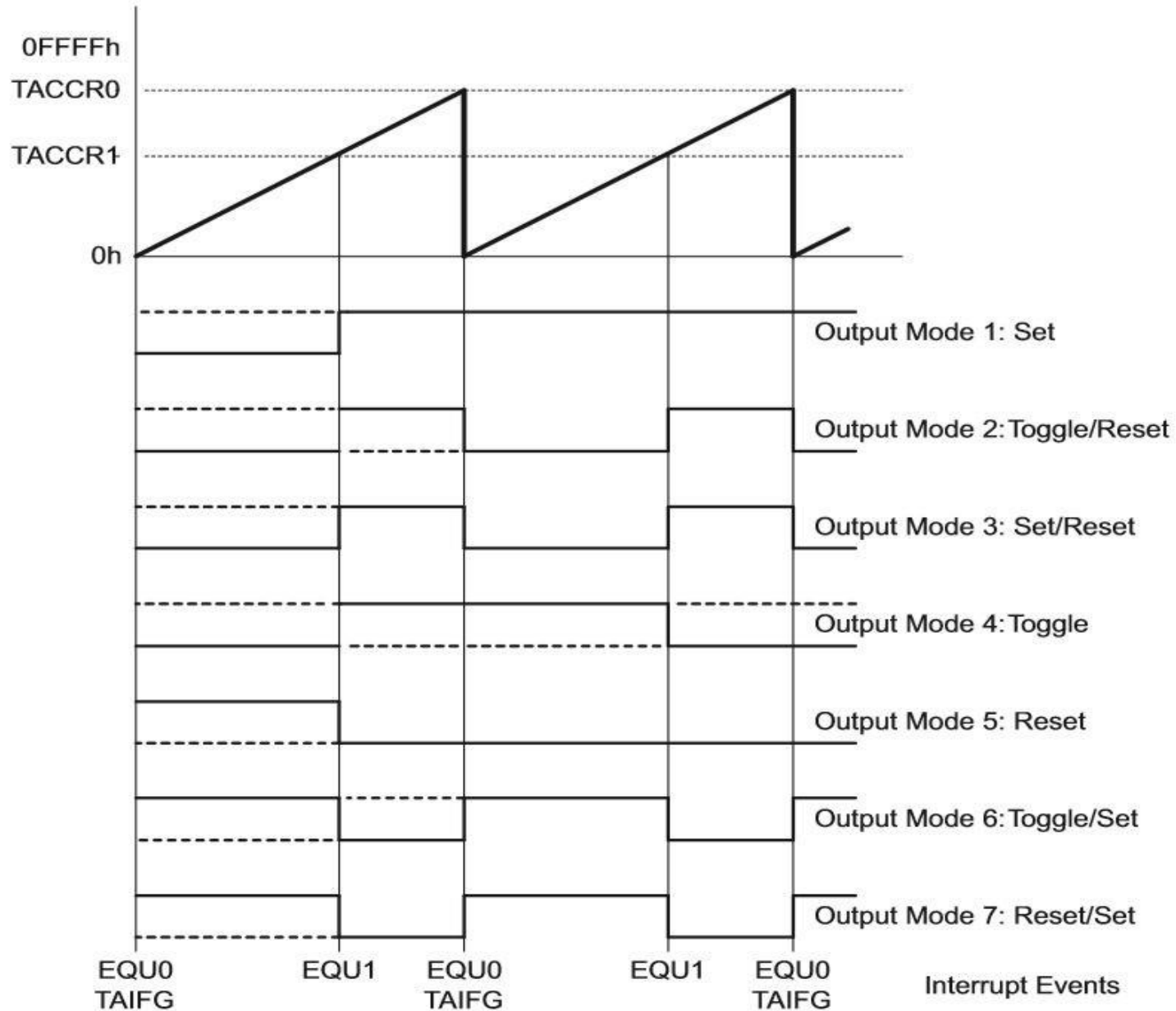
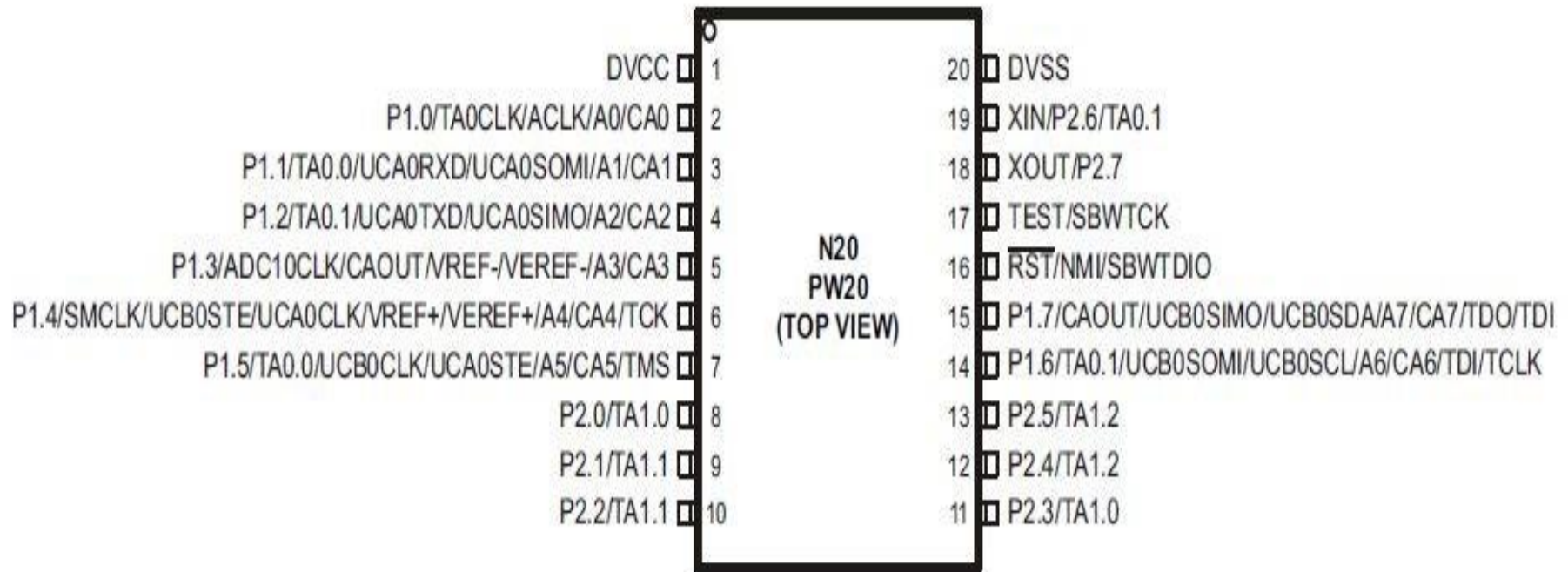


Figure 12-12. Output Example—Timer in Up Mode

PWM using Timer

Step 1: To implement a Timer output, a pin which has timer output functionality has to be chosen. Let us choose P1.6 as TA0.1 output of Timer0.



NOTE: ADC10 is available on MSP430G2x53 devices only.

NOTE: The pulldown resistors of port P3 should be enabled by setting P3REN.x = 1.

MSP430G2553 Timer1_A Input and Output Pins

You can find this information on Page 16 of the datasheet:

INPUT PIN NUMBER			DEVICE INPUT SIGNAL	MODULE INPUT NAME	MODULE BLOCK	MODULE OUTPUT SIGNAL	OUTPUT PIN NUMBER		
PW20, N20	PW28	RHB32					PW20, N20	PW28	RHB32
P1.0-2	P1.0-2	P1.0-31	TACLK	TACLK	Timer	NA			
			ACLK	ACLK					
			SMCLK	SMCLK					
PinOsc	PinOsc	PinOsc	TACLK	INCLK					
P1.1-3	P1.1-3	P1.1-1	TA0.0	CCI0A	CCR0	TA0	P1.1-3	P1.1-3	P1.1-1
			ACLK	CCI0B			P1.5-7	P1.5-7	P1.5-5
			V _{SS}	GND				P3.4-15	P3.4-14
			V _{CC}	V _{CC}					
P1.2-4	P1.2-4	P1.2-2	TA0.1	CCI1A	CCR1	TA1	P1.2-4	P1.2-4	P1.2-2
			CAOUT	CCI1B			P1.6-14	P1.6-22	P1.6-21
			V _{SS}	GND			P2.6-19	P2.6-27	P2.6-26
			V _{CC}	V _{CC}				P3.5-19	P3.5-18
	P3.0-9	P3.0-7	TA0.2	CCI2A	CCR2	TA2		P3.0-9	P3.0-7
PinOsc	PinOsc	PinOsc	TA0.2	CCI2B				P3.6-20	P3.6-19
			V _{SS}	GND					
			V _{CC}	V _{CC}					

MSP430G2553 Timer1_A Input and Output Pins

You can find this information on Page 16 of the datasheet:

INPUT PIN NUMBER			DEVICE INPUT SIGNAL	MODULE INPUT NAME	MODULE BLOCK	MODULE OUTPUT SIGNAL	OUTPUT PIN NUMBER		
PW20, N20	PW28	RHB32					PW20, N20	PW28	RHB32
-	P3.7-21	P3.7-20	TACLK	TACLK	Timer	NA			
			ACLK	ACLK					
			SMCLK	SMCLK					
-	P3.7-21	P3.7-20	TACLK	INCLK					
P2.0-8	P2.0-10	P2.0-9	TA1.0	CCI0A	CCR0	TA0	P2.0-8	P2.0-10	P2.0-9
P2.3-11	P2.3-16	P2.3-12	TA1.0	CCI0B			P2.3-11	P2.3-16	P2.3-15
			V _{SS}	GND				P3.1-8	P3.1-6
			V _{CC}	V _{CC}					
P2.1-9	P2.1-11	P2.1-10	TA1.1	CCI1A	CCR1	TA1	P2.1-9	P2.1-11	P2.1-10
P2.2-10	P2.2-12	P2.2-11	TA1.1	CCI1B			P2.2-10	P2.2-12	P2.2-11
			V _{SS}	GND				P3.2-13	P3.2-12
			V _{CC}	V _{CC}					
P2.4-12	P2.4-17	P2.4-16	TA1.2	CCI2A	CCR2	TA2	P2.4-12	P2.4-17	P2.4-16
P2.5-13	P2.5-18	P2.5-17	TA1.2	CCI2B			P2.5-13	P2.5-18	P2.5-17
			V _{SS}	GND				P3.3-14	P3.3-13
			V _{CC}	V _{CC}					

PWM Using Timer

STEP 1 : To select the timer function of that pin, PxDIR, PxSEL and PxSEL2 Registers are used.

PIN NAME (P1.x)	x	FUNCTION	CONTROL BITS AND SIGNALS ⁽¹⁾					
			P1DIR.x	P1SEL.x	P1SEL2.x	ADC10AE.x INCH.x=1 ⁽²⁾	JTAG Mode	CAPD.y
P1.5/ TA0.0/ UCB0CLK/ UCA0STE/ A5 ⁽²⁾ / CA5 TMS Pin Osc	5	P1.x (I/O)	I: 0; O: 1	0	0	0	0	0
		TA0.0	1	1	0	0	0	0
		UCB0CLK	from USCI	1	1	0	0	0
		UCA0STE	from USCI	1	1	0	0	0
		A5	X	X	X	1 (y = 5)	0	0
		CA5	X	X	X	0	0	1 (y = 5)
		TMS	X	X	X	0	1	0
		Capacitive sensing	X	0	1	0	0	0
P1.6/ TA0.1/ UCB0SOMI/ UCB0SCL/ A6 ⁽²⁾ / CA6 TDI/TCLK/ Pin Osc	6	P1.x (I/O)	I: 0; O: 1	0	0	0	0	0
		TA0.1	1	1	0	0	0	0
		UCB0SOMI	from USCI	1	1	0	0	0
		UCB0SCL	from USCI	1	1	0	0	0
		A6	X	X	X	1 (y = 6)	0	0
		CA6	X	X	X	0	0	1 (y = 6)
		TDI/TCLK	X	X	X	0	1	0
		Capacitive sensing	X	0	1	0	0	0

PWM Using Timer

Step 2: Timer is configured in Up mode, so TAR value goes from 0 to TACCR0 and repeats. Up mode is selected by ORing 'MC_1' to TACTL register.

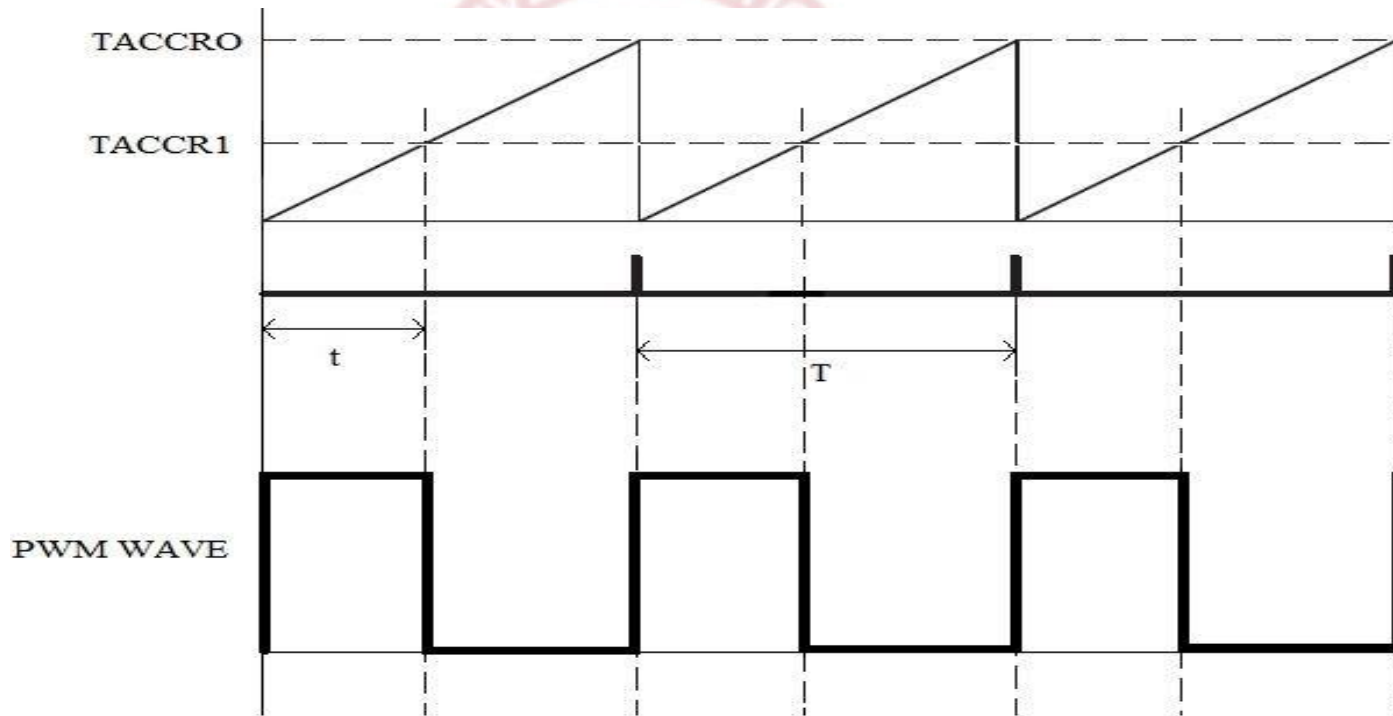
Step 3: The value of TACCR0 register is filled. TACCR0 sets time period (T) of PWM.

Step 4: The value of TACCR1 register is filled. TACCR1 sets the on-time (t) of PWM.

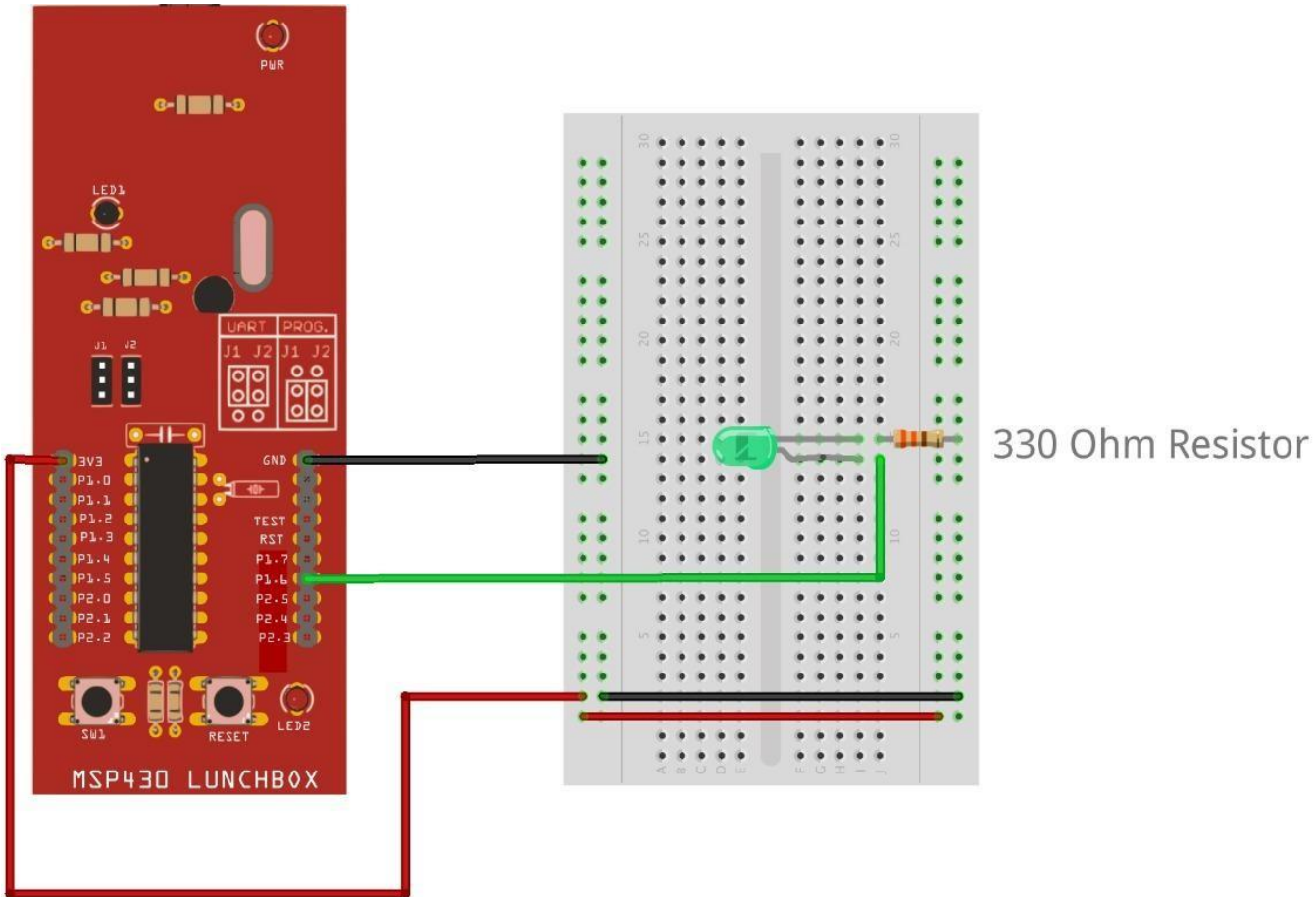
Step 5: Output mode 7 is selected by writing 'OUTMOD_7' to TACCTL1 register : $TACCTL1 = OUTMOD_7$;

PWM using Timer

- Output resets when timer (TAR) value counts to TACCR1 value.
- Output value sets when timer (TAR) value count reaches TACCR0 value.



Circuit Diagram



fritzing

1.1 MHz

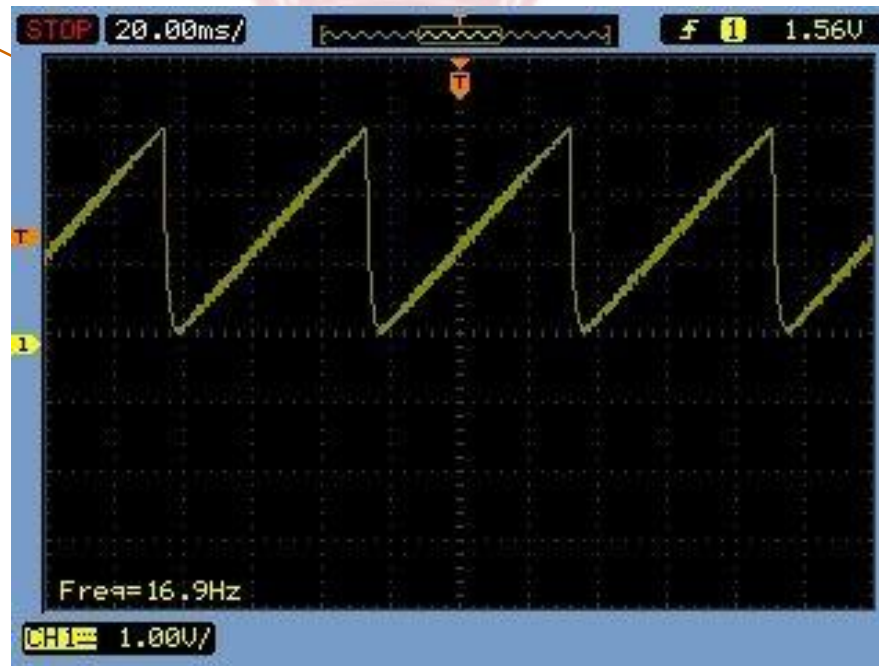
```
1 #include <msp430.h>
2
3 #define GREEN    BIT6           // Green LED -> P1.6
4
5 /**
6  * @brief
7  * These settings are w.r.t enabling TIMER0 on Lunchbox
8  */
9 void register_settings_for_TIMER0()
10 {
11     P1DIR |= GREEN;             // Green LED -> Output
12     P1SEL |= GREEN;             // Green LED -> Select Timer Output
13
14     CCR0 = 255;                 // Set Timer0 PWM Period
15     CCTL1 = OUTMOD_7;           // Set TA0.1 Waveform Mode - Clear on Compare, Set on Overflow
16     CCR1 = 0;                   // Set TA0.1 PWM duty cycle
17     CCTL0 = CCIE;               // CCR0 Enable Interrupt
18     TACTL = TASSEL_2 + MC_1;    // Timer Clock -> SMCLK, Mode -> Up Count
19 }
20
```

```

20
21 /**
22  * @brief
23  * Entry point for the code
24  */
25 void main(void) {
26
27     WDTCTL = WDTPW | WDTHOLD;           // Stop watchdog timer
28
29     register_settings_for_TIMER0();
30
31     __bis_SR_register(GIE);             // Enable CPU Interrupt
32
33     while(1)
34     {
35     }
36
37 }
38
39 /**
40  * @brief
41  * Entry point for TIMER0_interrupt vector
42  */
43 #pragma vector = TIMER0_A0_VECTOR
44 __interrupt void Timer_A(void)
45 {
46     CCR1 = CCR1 + 1;                     // Increment CCR1
47     if(CCR1 == 256)
48     {
49         CCR1 = 0;
50     }
51 }

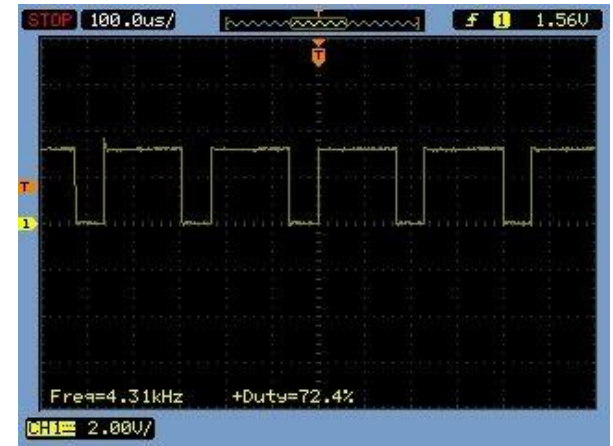
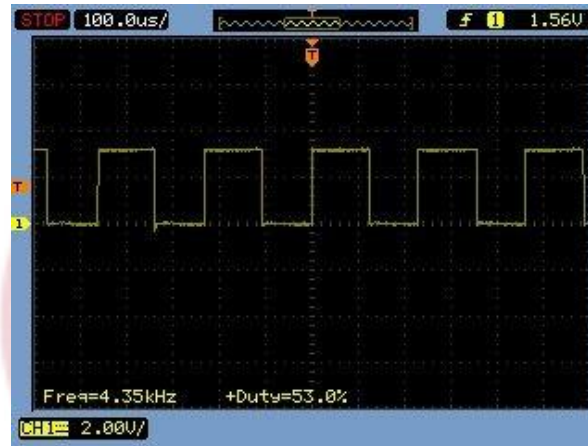
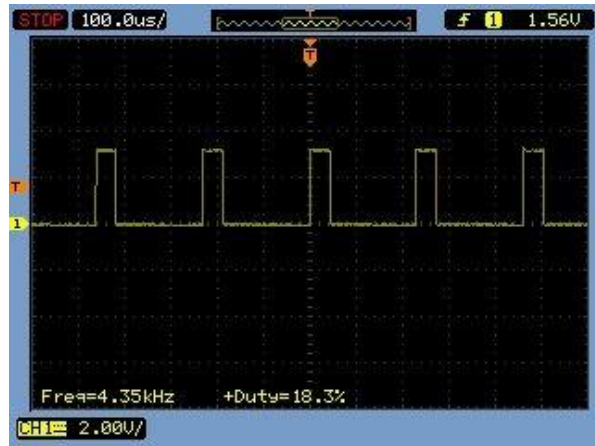
```

Filtered Output



- $R = 470 \text{ ohm}$, $C = 2.2 \text{ uF}$
- Filter Frequency = $1/(2 \times 3.14 \times R \times C)$

PWM signal at pin 1.6



Hardware PWM Code 2 16 bit PWM at DCO freq - 16 MHz



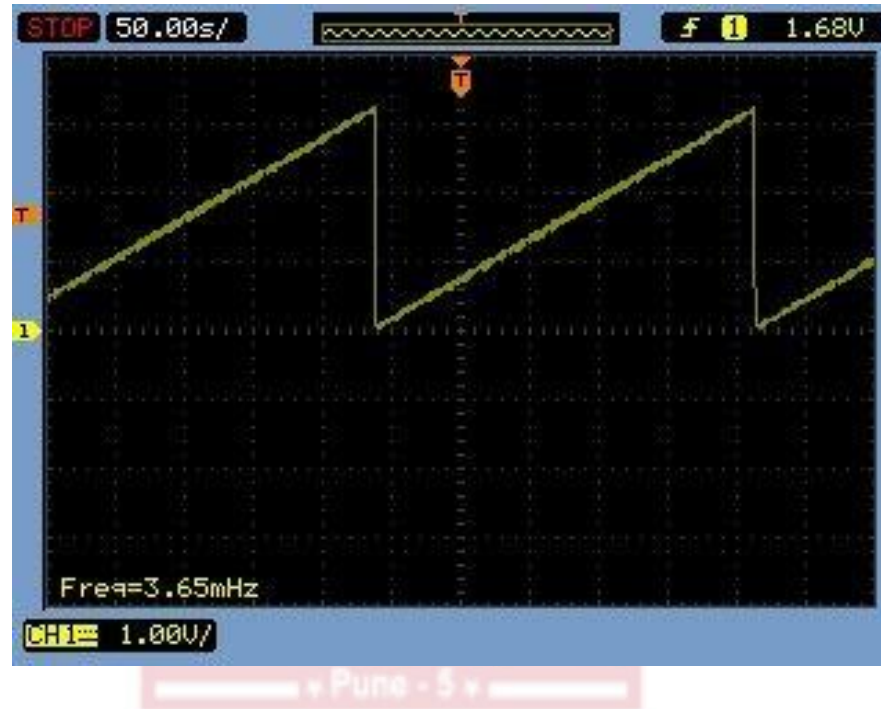
```
1 #include <msp430.h>
2
3 #define GREEN    BIT6           // Green LED -> P1.6
4
5 /**
6  * @brief
7  * These settings are w.r.t enabling TIMER0 on Lunchbox
8  */
9 void register_settings_for_TIMER0()
10 {
11     P1DIR |= GREEN;             // Green LED -> Output
12     P1SEL |= GREEN;             // Green LED -> Select Timer Output
13
14     CCR0 = 65535;                // Set Timer0 PWM Period
15     CCTL1 = OUTMOD_7;            // Set TA0.1 Waveform Mode - Clear on Compare, Set on Overflow
16     CCR1 = 0;                    // Set TA0.1 PWM duty cycle
17     CCTL0 = CCIE;                // CCR0 Enable Interrupt
18     TACTL = TASSEL_2 + MC_1;     // Timer Clock -> SMCLK, Mode -> Up Count
19 }
20
```

```

20
21 /**
22  * @brief
23  * Entry point for the code
24  */
25 void main(void) {
26
27     WDTCTL = WDTPW | WDTHOLD;           // Stop watchdog timer
28     BCSCCTL1 |= (BIT0 + BIT1 + BIT2 + BIT3); // Selecting RSELx as 15
29     DCOCTL  |= (BIT6 + BIT5 + BIT4);      // Selecting DCOX as 7, DCO_freq = 15.6 MHz, Room Temp. ~ 25 deg. Celsius, Operating voltage 3.3 V
30
31     register_settings_for_TIMER0();
32
33     __bis_SR_register(GIE);              // Enable CPU Interrupt
34
35     while(1)
36     {
37     }
38
39 }
40
41 /**
42  * @brief
43  * Entry point for TIMER0_interrupt vector
44  */
45 #pragma vector = TIMER0_A0_VECTOR
46 __interrupt void Timer_A(void)
47 {
48     CCR1 = CCR1 + 1;                    // Increment CCR1
49 }

```

Filtered Output

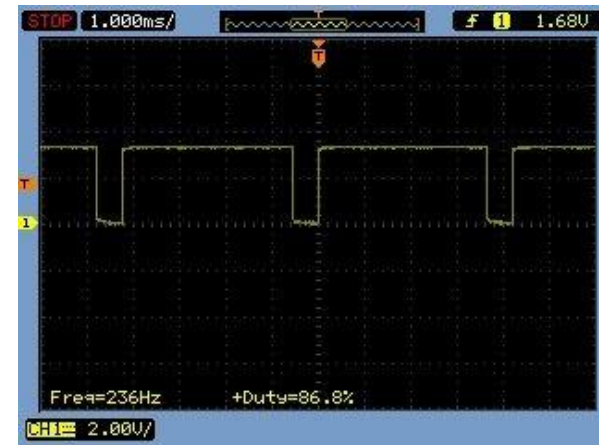
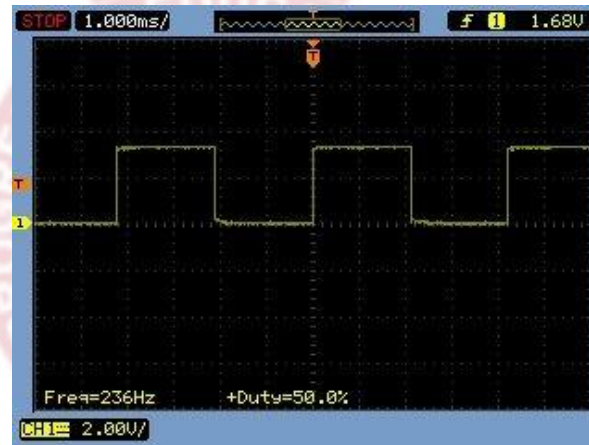
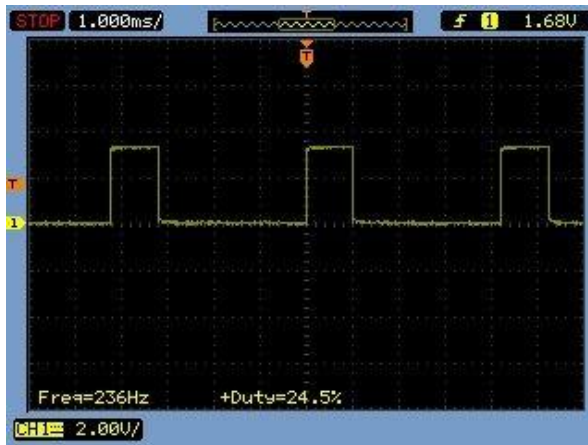


$R = 2.2 \text{ Kohm}$, $C = 22 \text{ uF}$

Filter Frequency = $1/(2 \times 3.14 \times R \times C) = 3.28 \text{ Hz}$

PWM signal at pin 1.6

Duty cycle of 24.5 %, 50 %, 86.8 %



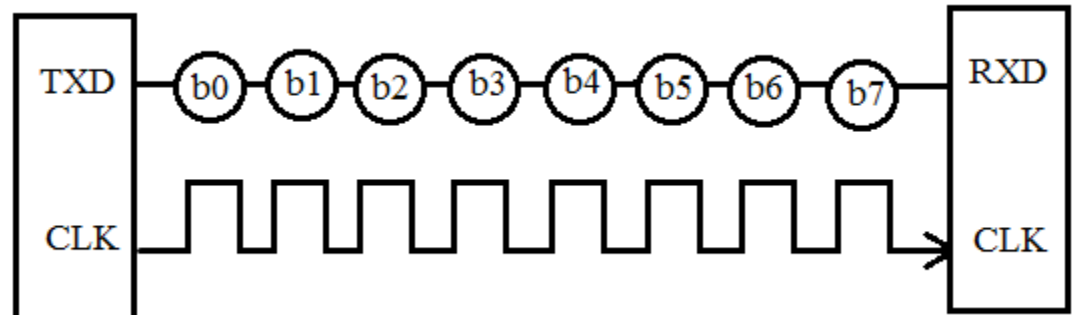
Modern College of Engineering
Pune - 5

Basics of Serial Communication Protocol

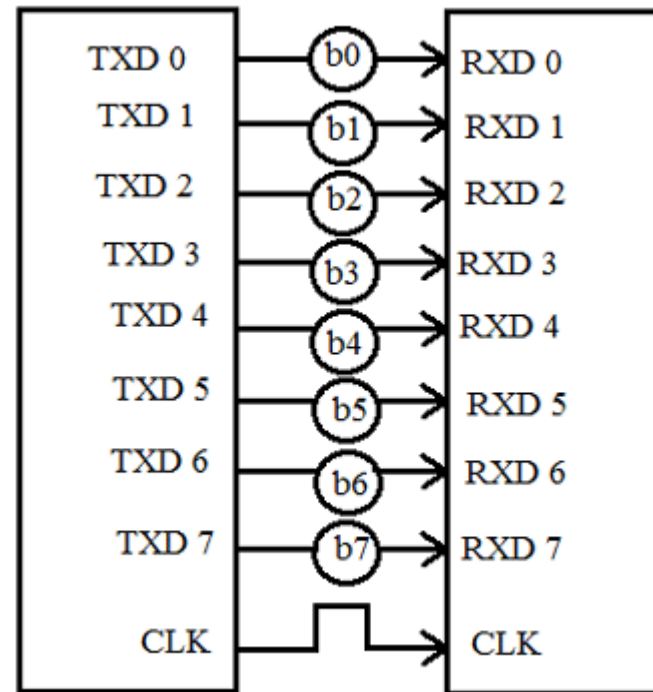


Types of Communication Protocols

- A protocol is the secure and reliable form of communication having a set of rules addressed by the source host (sender) and destination host (receiver).
- Serial Communication Protocols: Eg: CAN, ETHERNET, I2C, SPI, RS232, USB, 1-Wire and SATA.
- Parallel Communication Protocols: Eg: ISA, ATA, SCSI, PCI and IEEE-488.



Serial Communication Protocol



Parallel Communication Protocols

Serial Communications

- Serial communications is used extensively within the electronics industry due to its relative simplicity and low hardware overhead (as compared to parallel interfacing).
- EIA/TIA-232-E specification is the most popular serial communications standard.
- This standard, developed by the Electronic Industry Association and the Telecommunications Industry Association (EIA/TIA), is more popularly referred to simply as “RS232” where “RS” stands for “Recommended Standard.”
- The EIA-232 interface standard was developed for interfacing data terminal equipment (DTE) and data circuit terminating equipment (DCE) (earlier called as Data Communication Equipment) employing serial binary data interchange.
- EIA-232 was particularly developed for interfacing data terminals to modems.

Serial Communication Protocols

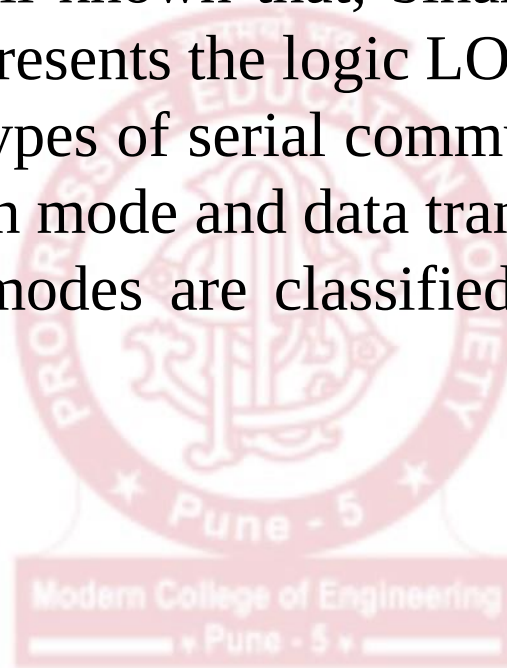
- In embedded systems, communication means the exchange of data between two microcontrollers in the form of bits.
- This exchange of data bits in microcontroller is done by some set of defined rules known as **communication protocols**.
- Now if the data is sent in series i.e. one after the other then the communication protocol is known as **Serial Communication Protocol**.
- More specifically, the data bits are transmitted one at a time in sequential manner over the data bus or communication channel in Serial Communication.
- There are a very large number of different standards and protocols for serial communication, ranging from the very simple to the seriously complicated.
- It is important to match the right protocol with the right application.

Types of Serial Communications Protocols

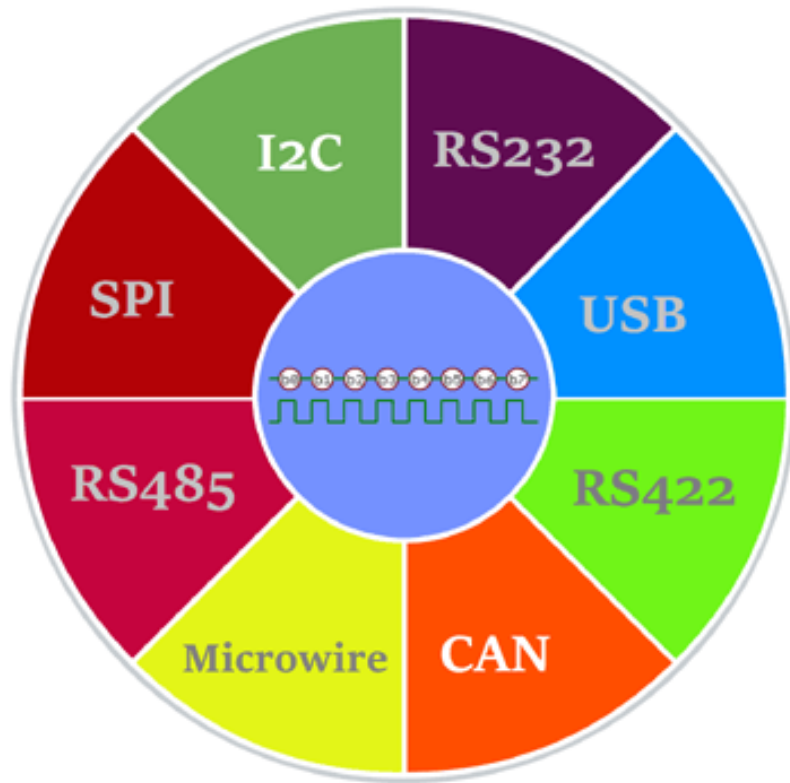
- There are two broad types of serial communication: **synchronous and asynchronous**.
- The synchronous type of serial protocols such as SPI, I2C, CAN and LIN.
- The asynchronous type of serial protocols are very essential when it comes to longer distance reliable data transfer.
- Asynchronous communication does not require a timing clock that is common to both devices.
- Each device independently listens and sends digital pulses that represent bits of data at an agreed-upon rate.
- The examples are RS232, RS422, RS485 etc.

Transmission Modes in Serial Communication

- In serial communication data is sent in the form of bits i.e. binary pulses and it is well known that, binary one represents the logic HIGH and zero represents the logic LOW.
- There are several types of serial communication depending on the type of transmission mode and data transfer.
- The transmission modes are classified as **Simplex, Half Duplex and Full Duplex**.



Serial Communication Protocols



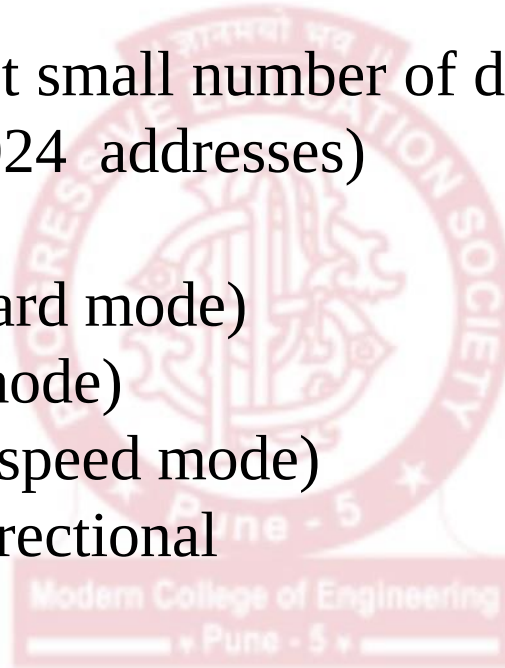
Serial Communication Protocols



I2C

I2C Serial Communication

- Inter Integrated Circuits (I2C) bus by Philips Semiconductors, two-wired bus
- Originally to interact small number of devices, but nowadays 10 bit addressing (up to 1024 addresses)
- Speeds:
 - 100 kbps (standard mode)
 - 400 kbps (fast mode)
 - 3.4 Mbps (high-speed mode)
- Data transfers: bi-directional
- Multi-master option

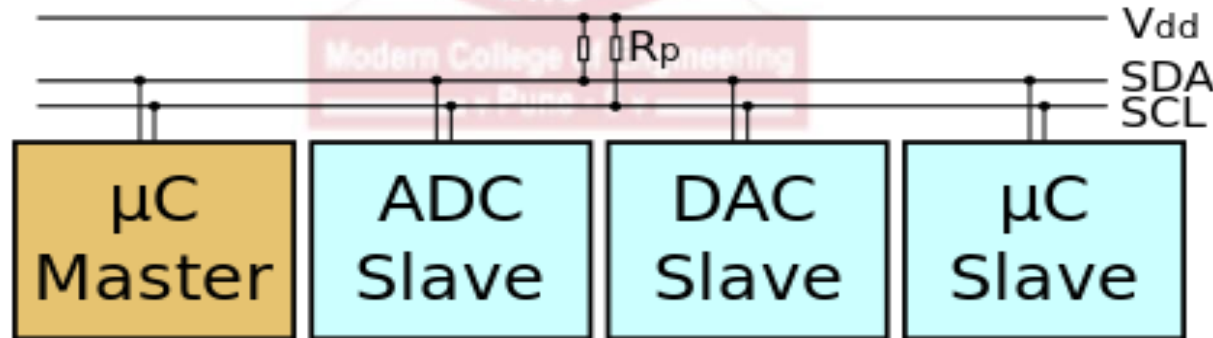


I2C Serial Communication

- Inter integrated circuit (I2C) two-line communication between different ICs or modules where two lines are SDA (Serial Data Line) and SCL (Serial Clock Line).
- Both the lines must be connected to a positive supply using a pull up resistor.
- I2C can deliver speed up to 400Kbps and it uses 10 bit or 7 bit addressing system to target a specific device on the i2c bus so it can connect up to 1024 devices.
- It has limited length communication and is ideal for onboard communication.
- I2C networks are easy to setup since it uses only two wires and new devices can simply be connected to the two common I2C bus lines.
- Same like SPI, microcontroller generally have I2C pins to connect any I2C device.

I2C Serial Communication

- A master device sends the sequence of signal with START, ADDRESS, and WRITE protocol, then waits for an acknowledge bit (A) from the slave
- The slave will generate acknowledge bit only if its internal address matches the value sent by the master.
- After that the master sends DATA and waits for acknowledge (A) from the slave. The master completes the byte transfer by generating a stop bit (P).

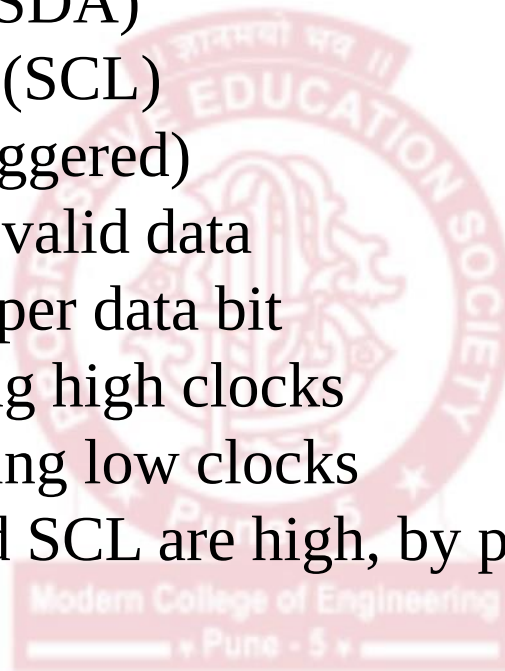


I2C Hardware

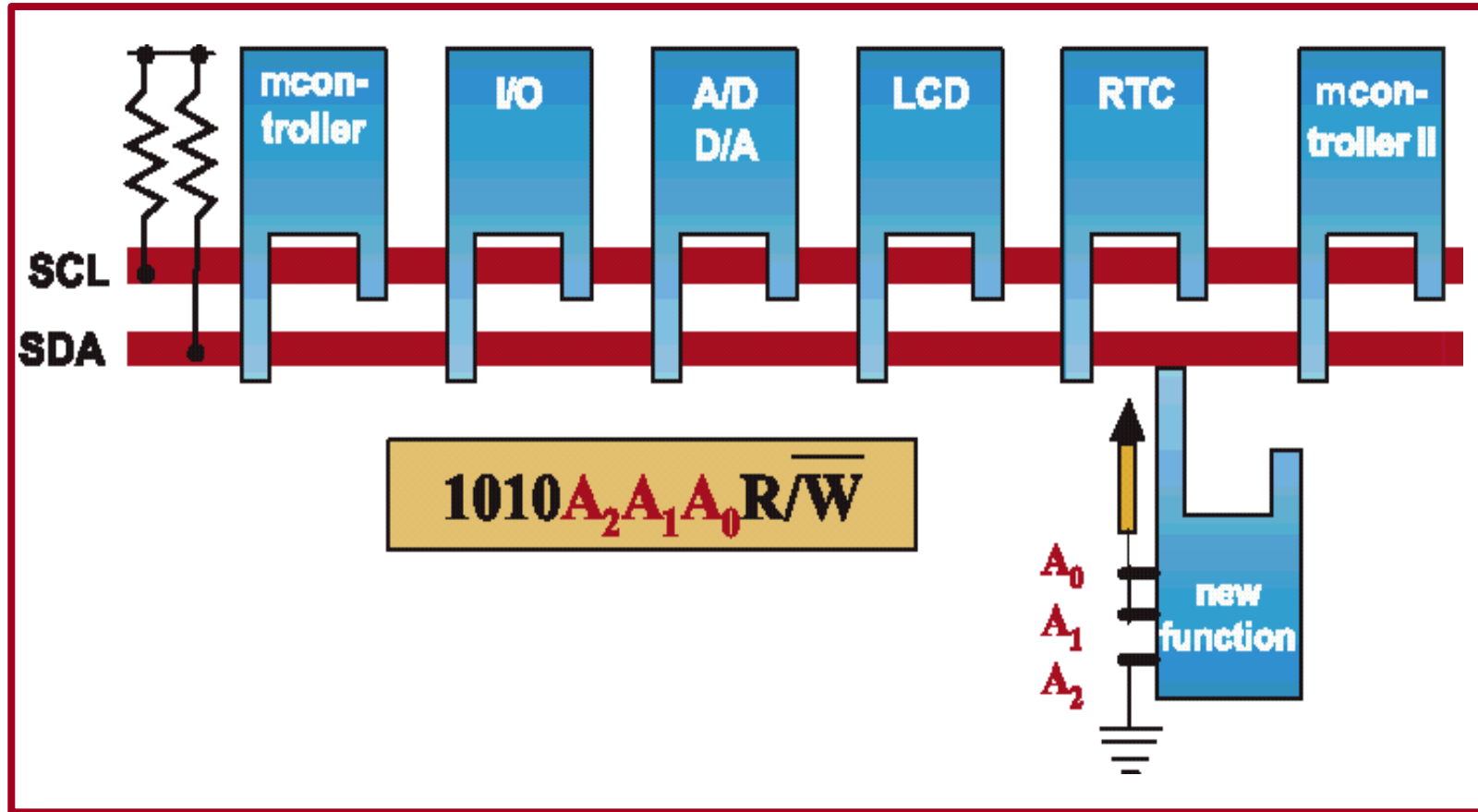
- **Transmitter** – The device sending data to the bus
- **Receiver** – Device receiving data from the bus
- **Master** – device initiating a transfer, generates the clock and terminates a transfer, Transmits the slave address.
- **Slave** – Responds only when addressed.
- Devices connected to the bus must have an open drain or open collector output for serial clock and data.
- The data transfer is Bidirectional (Half Duplex).
- **Multi-master** – more than one master can attempt to control the bus
- **Arbitration** – procedure to ensure that only one master has control of there bus at any instant.
- The serial clock and data lines are connected to VCC through pull up resistors.

Wires and Signals

- Two-wired bus
 - serial data line (SDA)
 - serial clock line (SCL)
- bit transfer (level triggered)
 - $SCL = 1$ SDA = valid data
 - one clock pulse per data bit
 - stable data during high clocks
 - data change during low clocks
- bus is free, SDA and SCL are high, by pull-up resistors



I2C Serial Connection



R/W = 1 : READ

R/W = 0 : WRITE

Data transfer sequence

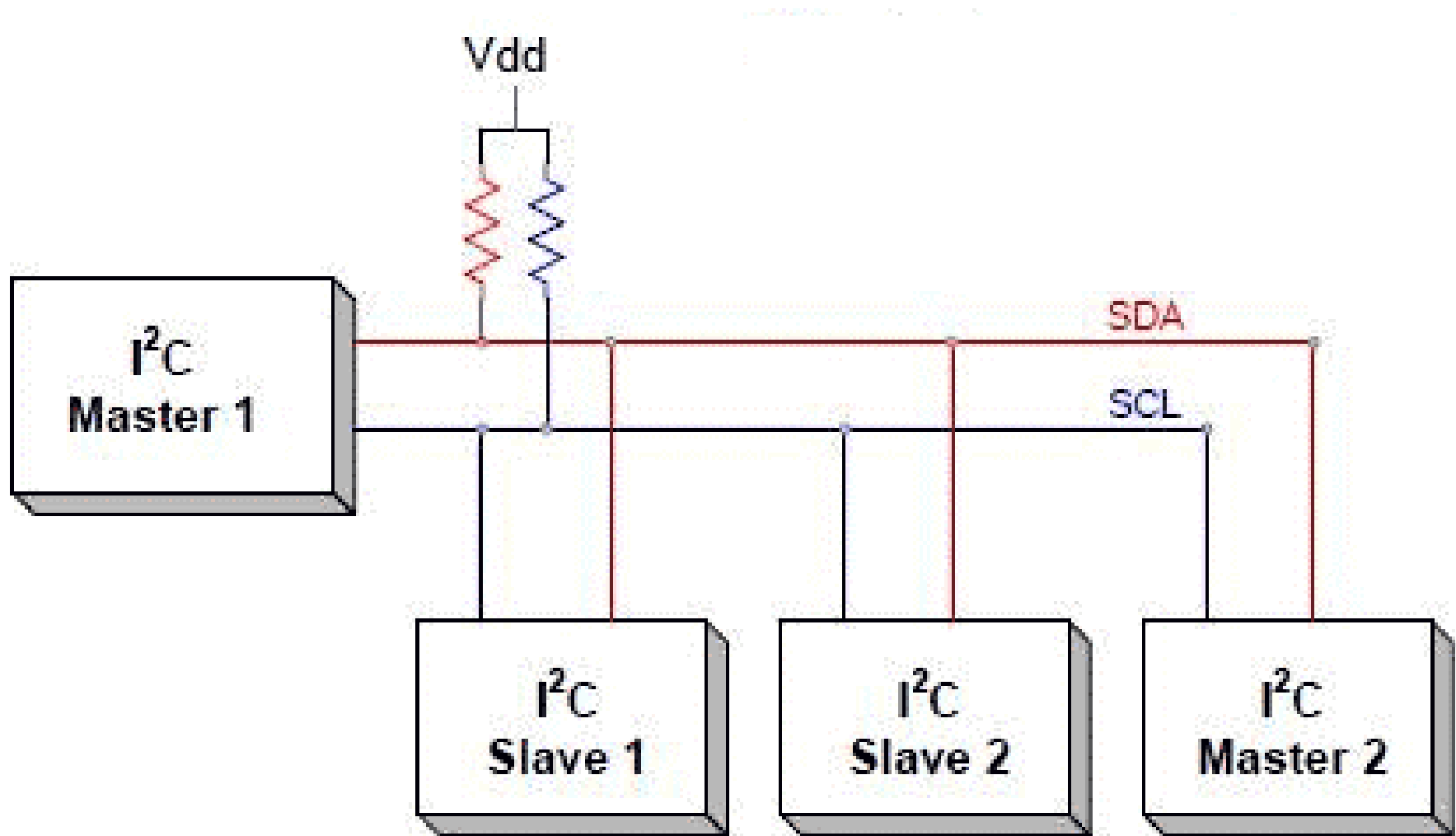
1. Transfers are byte oriented, MSB first
2. Start: SDA goes low while SCL is high
3. Data bits are transferred after start condition, most significant bit (MSB) first.
4. Master sends address of slave (7-bits) on next 7 clocks
5. Master sends read/write request bit
 1. 0-write to slave
 2. 1-read from slave
6. Slave ACKs by pulling SDA low on next clock

Data transfer sequence

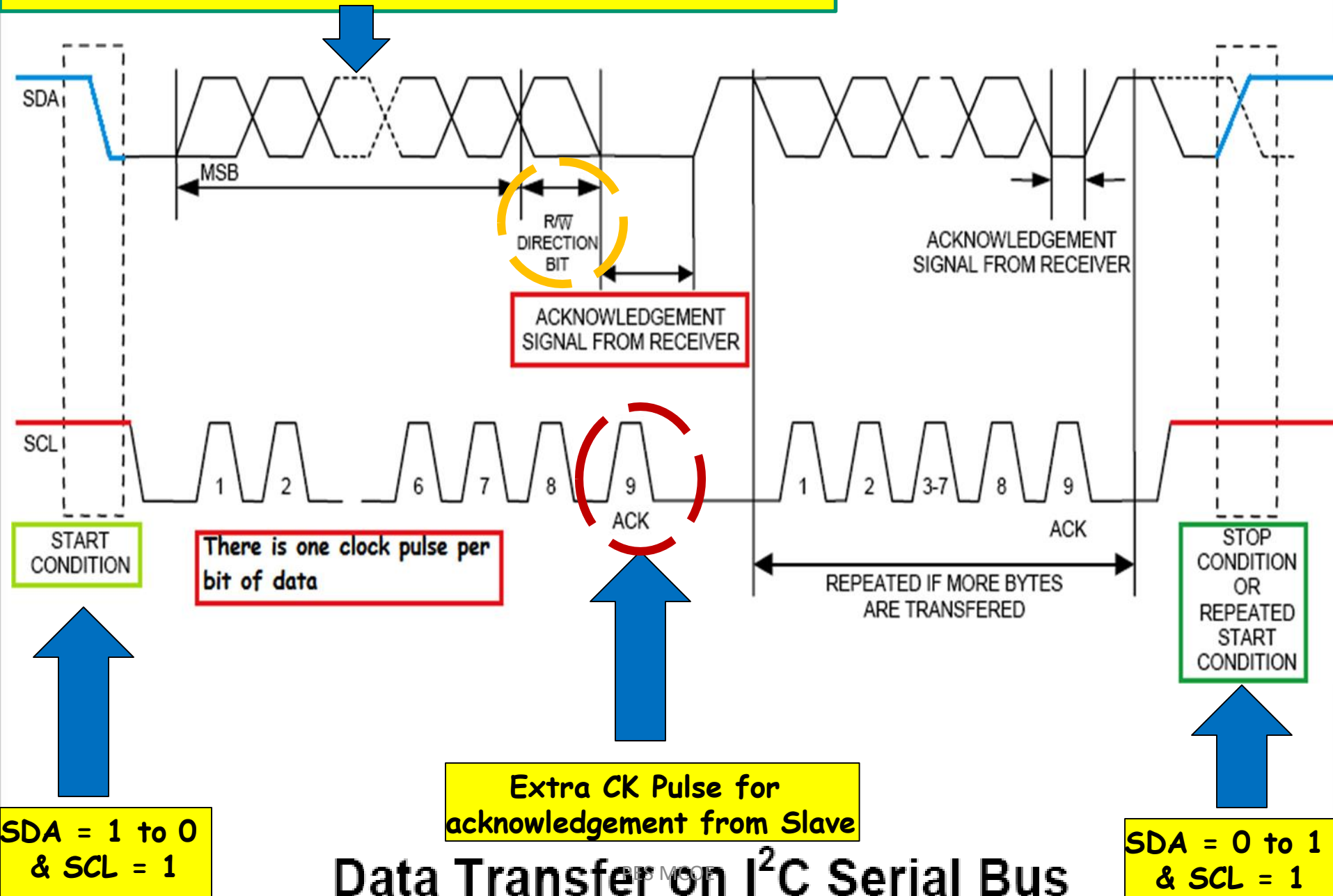
A Master to slave read or write sequence for I2C follows the following order:

1. Send the START bit (S).
2. Send the address (ADDR).
3. Send the Read(R)-1 / Write (W)-0 bit.
4. Wait for/Send an acknowledge bit (A).
5. Send/Receive the data byte (8 bits) (DATA).
6. Expect/Send acknowledge bit (A).
7. Send STOP bit (P).

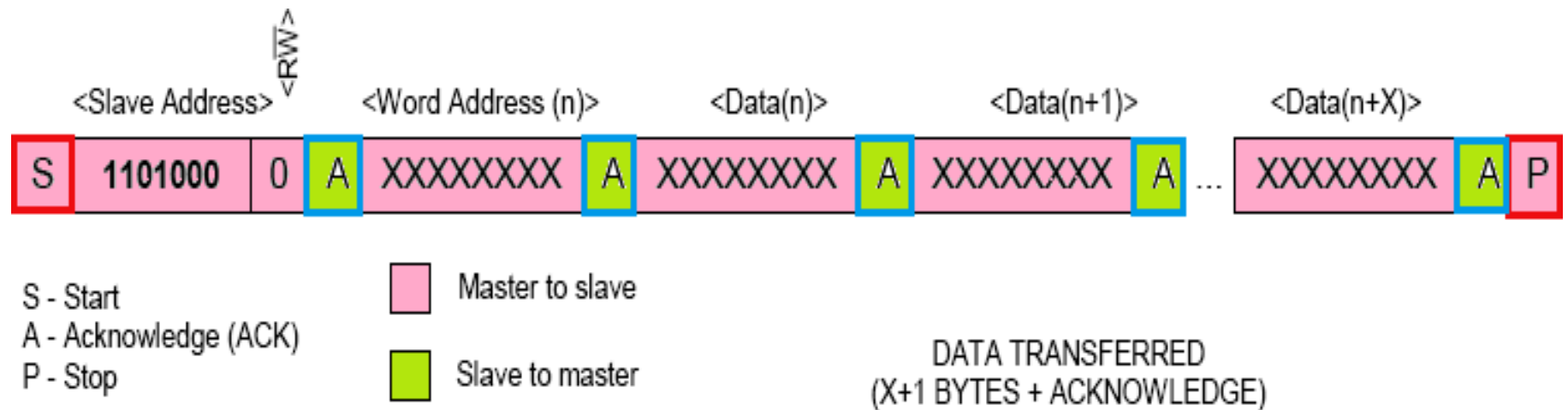
I²C Protocol



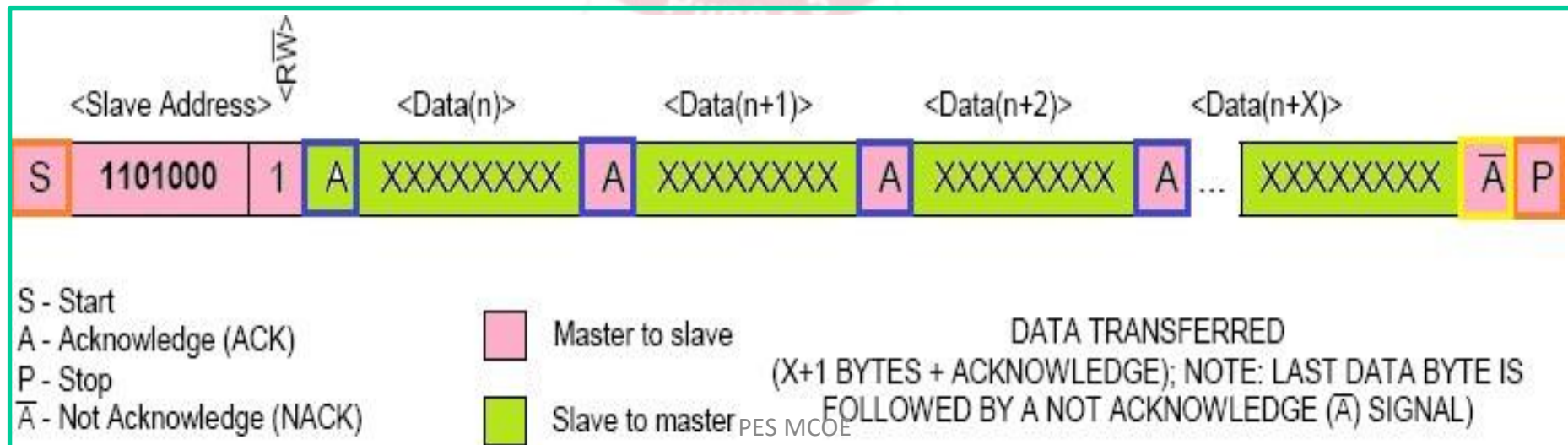
The data on the line must be changed during the LOW period of the clock signal



Data Write—Slave Receiver Mode



Data Read—Slave Transmitter Mode





SPI

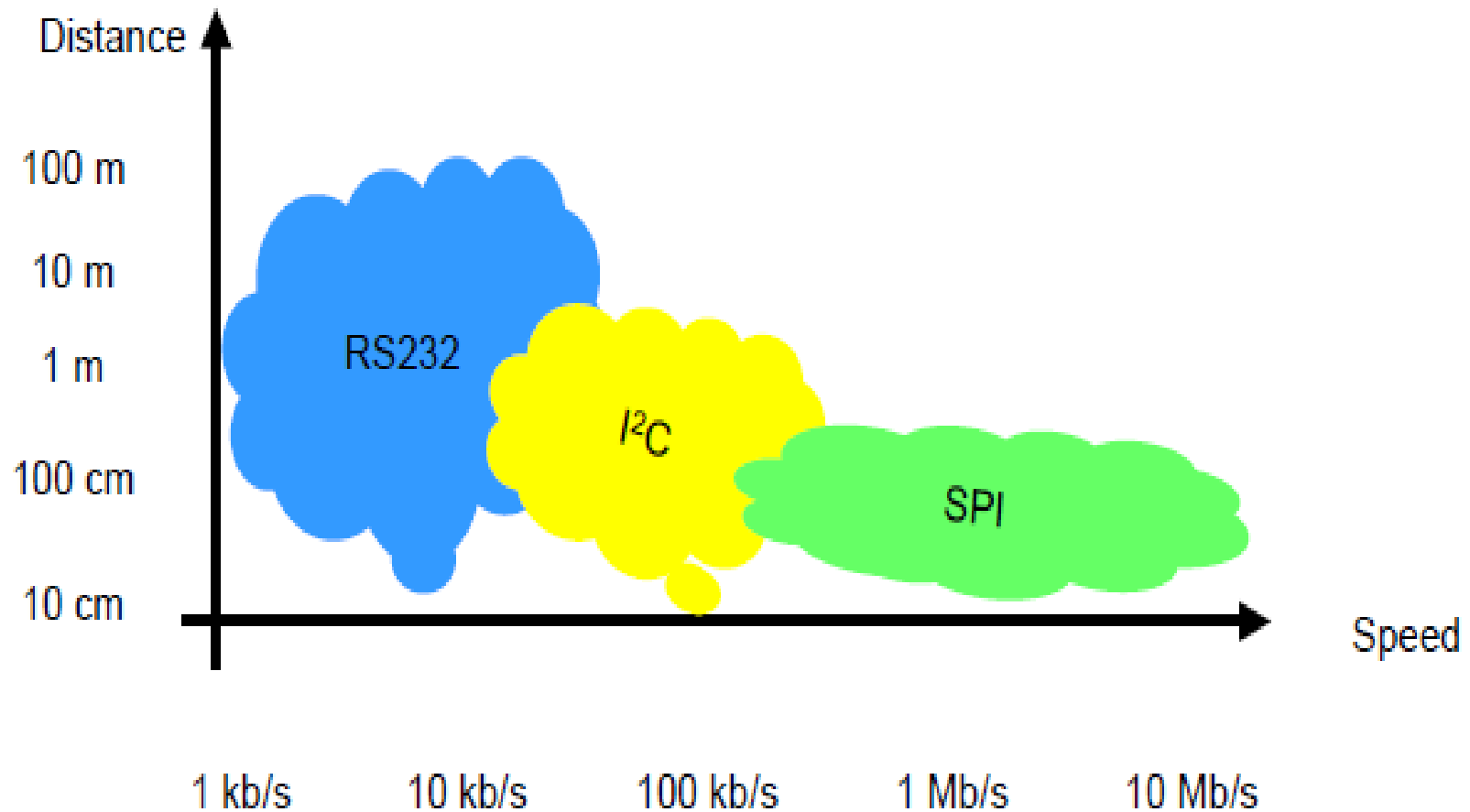
SPI Protocol

- Developed by Motorola
- Synchronous Serial Communication
- Used for serial data transfer up to 3 megabits/s Has data word size of 8 bits
- Data can be transferred on multiple bytes –pages or blocks
- Devices can be- Converters (ADC, DAC)
 - Memories (EEPROM, RAM's,Flash)
 - Sensors (Temperature, Humidity, Pressure)
 - Real Time Clocks, USB controllers, CAN controllers

SPI Protocol

- The Serial Peripheral Interface (SPI) is a synchronous interface which allows several SPI microcontrollers to be interconnected.
- In SPI, separate wires are required for data and clock line.
- Also, the clock is not included in the data stream and must be furnished as a separate signal.
- The SPI may be configured either as master or as a slave.
- The four basic SPI signals (MISO, MOSI, SCK and SS), Vcc and Ground are the part of data communication.
- So, it needs 6 wires to send and receive data from slave or master.
- Theoretically, the SPI can have unlimited number of slaves.
- The data communication is configured in SPI registers.
- The SPI can deliver up to 10Mbps of speed and is ideal for high-speed data communication.

Communication Protocols

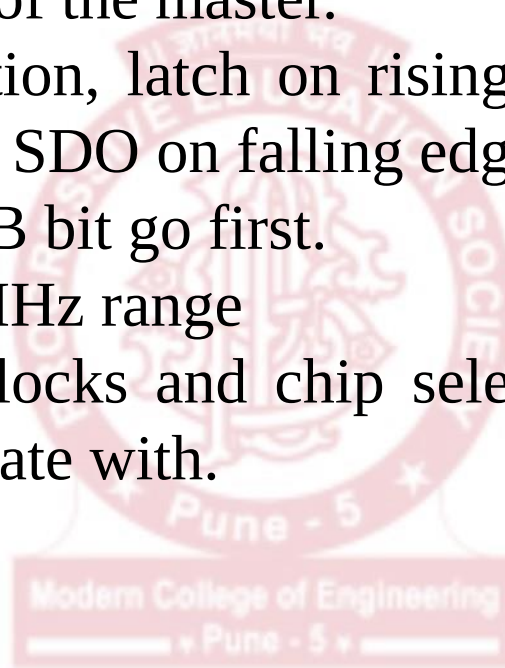


SPI Bus

- It first send data byte instruction to receiver.
- It then sends 16 bit address, Followed by 8 bits of I/O.
- It has symmetrical clock.
- Output data should be available at least 30 ns before clock goes high and read 30 ns before falling edge of clock.
- While writing data combine DO and DI pin with careful indications for I/O.
- For SPI, there are Serial Clocks (SCLK), Chip Select lines (CS), Serial Data In (SDI) and Serial Data Out(SDO)
- SPI devices use only 2 pins for data transfer: SDI & SDO.
- SCLK: Pin used for synchronization.
- CE: Chip enable pin used for initiate and terminate the data transfer.

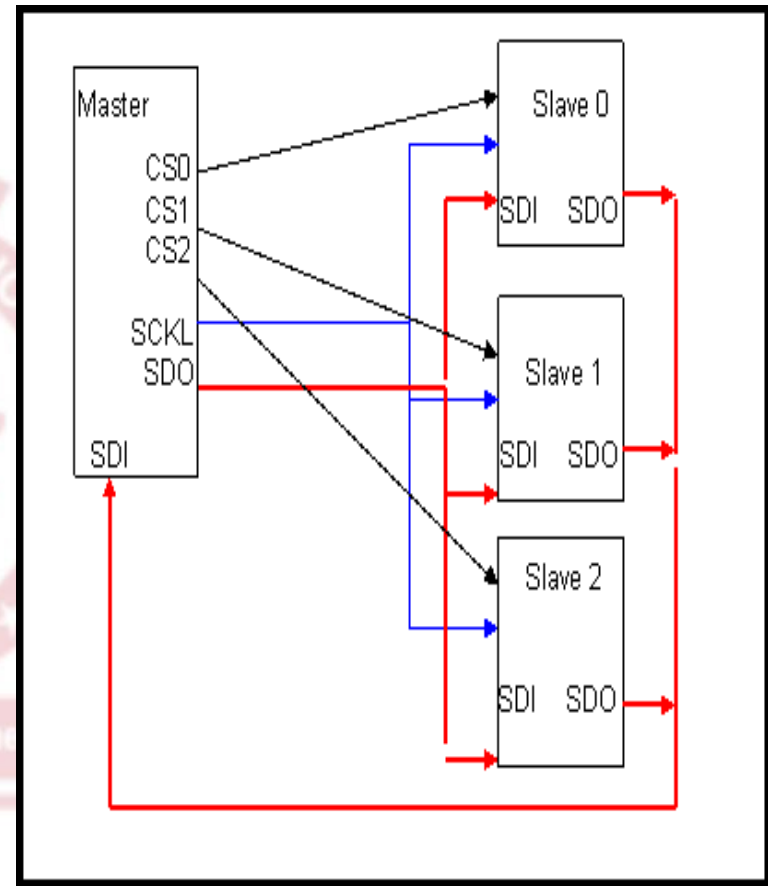
SPI Bus

- There is only one master, number of slaves depends on the number of chip select lines of the master.
- Synchronous operation, latch on rising or falling edge of clock, SDI on rising edge, SDO on falling edge
- In SPI Protocol MSB bit go first.
- Operates in 1 to 2 MHz range
- Master sends out clocks and chip selects. Activates the slaves it wants to communicate with.



Master Slave Setup

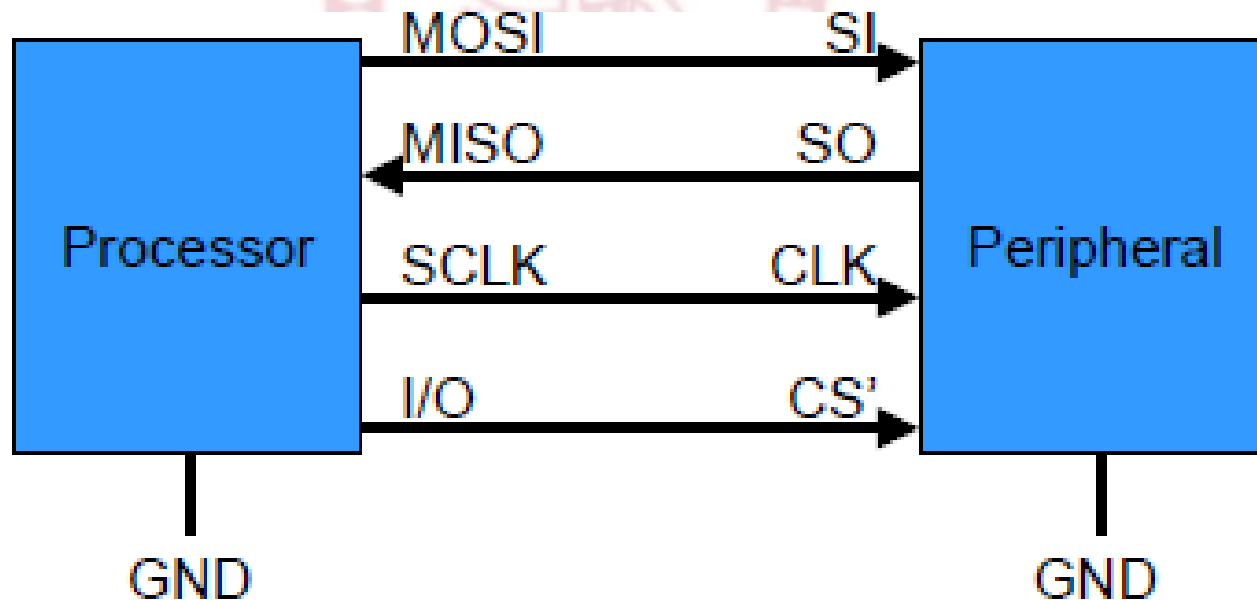
- In this setup, there are 3 slave devices.
- The SDO lines are tied together to the SDI line of the master.
- The master determines which chip it is talking to by the CS lines.
- For the slaves that are not being talked to, the data output goes to a Hi Z state.



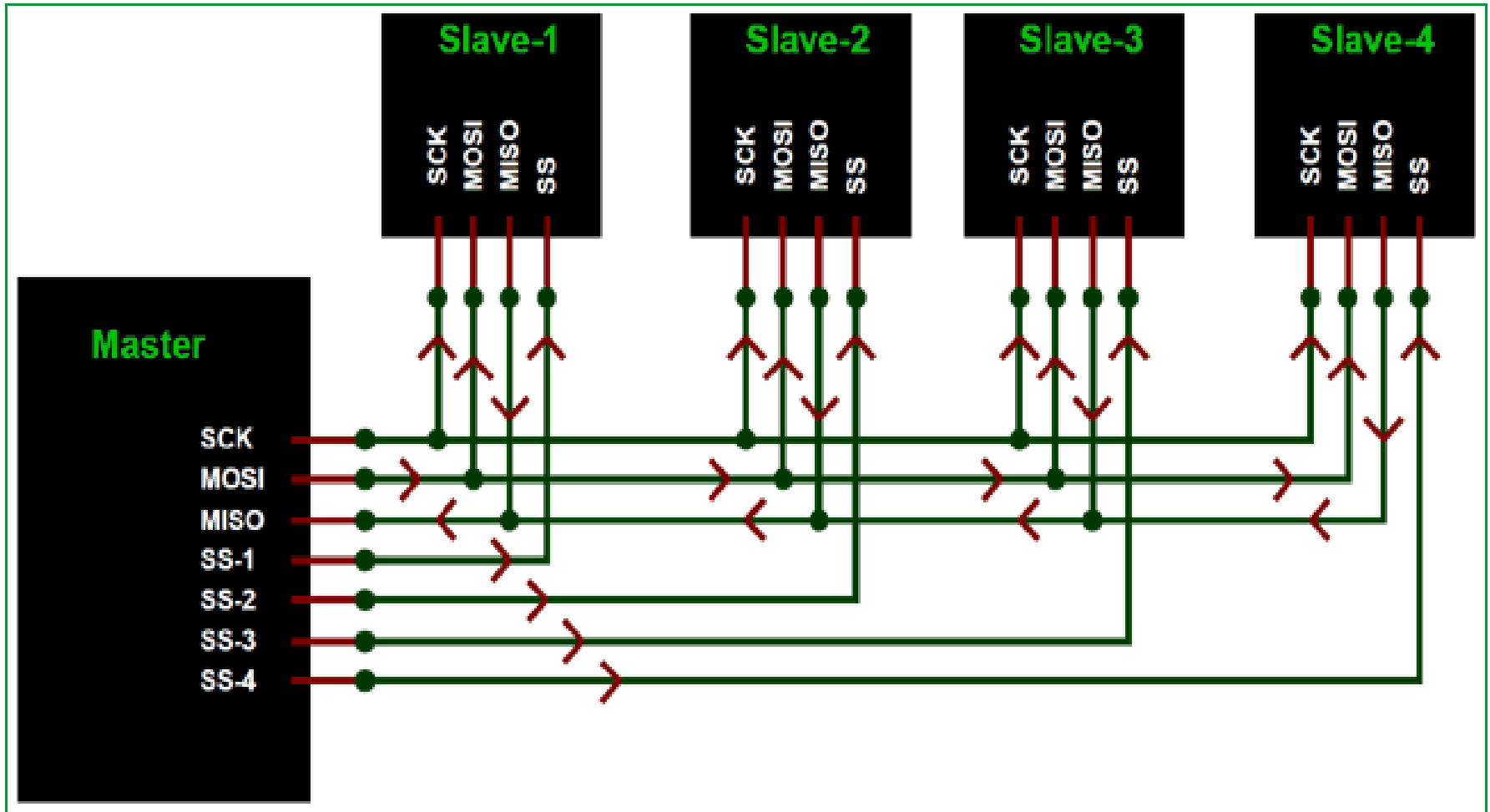
SPI Bus

SPI is a Master-Slave protocol:

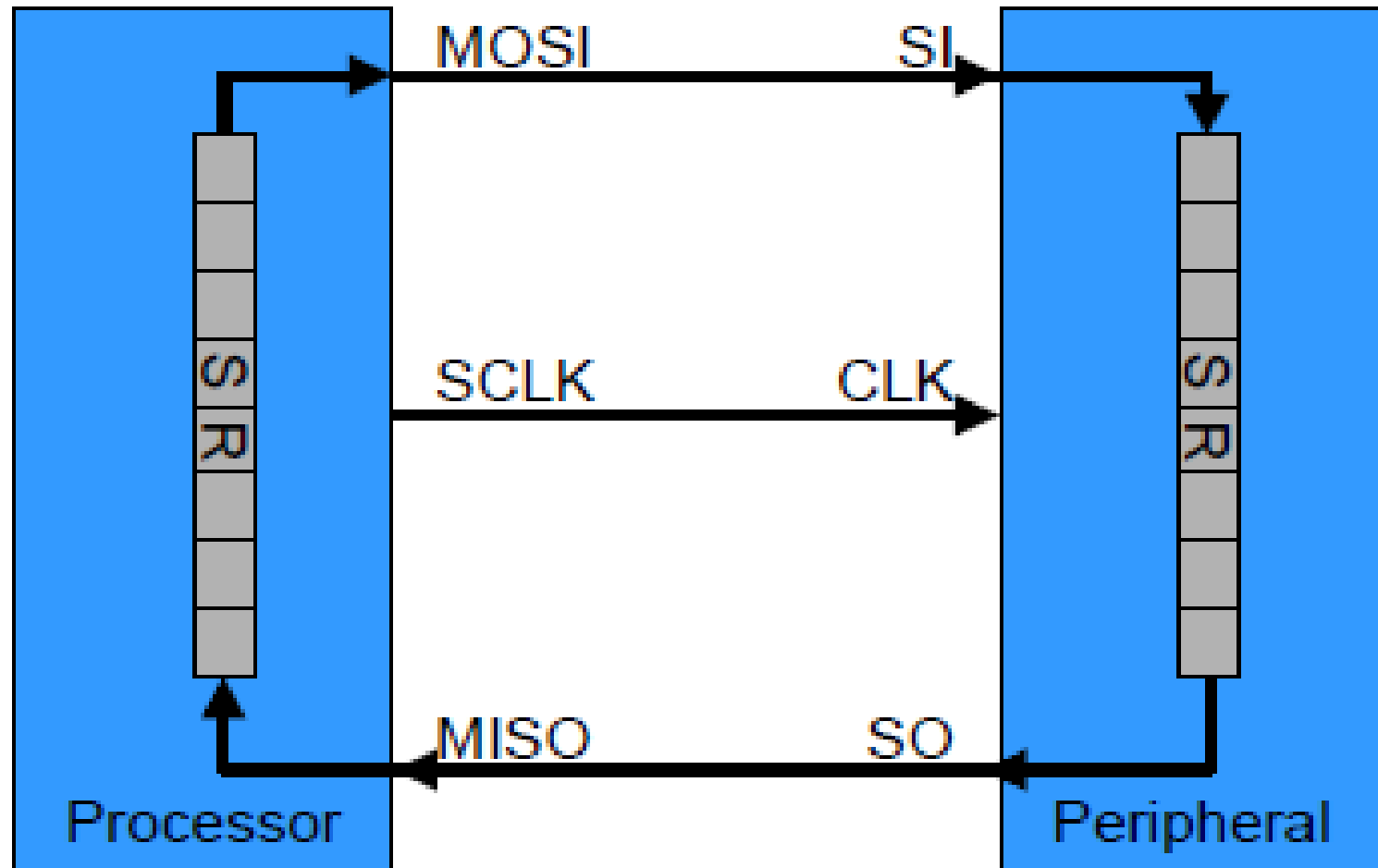
- The Master device controls the clock (SCK)
- No data is transferred unless a clock signal is present
- All slaves are controlled by the master clock
- The slave devices may not manipulate the clock



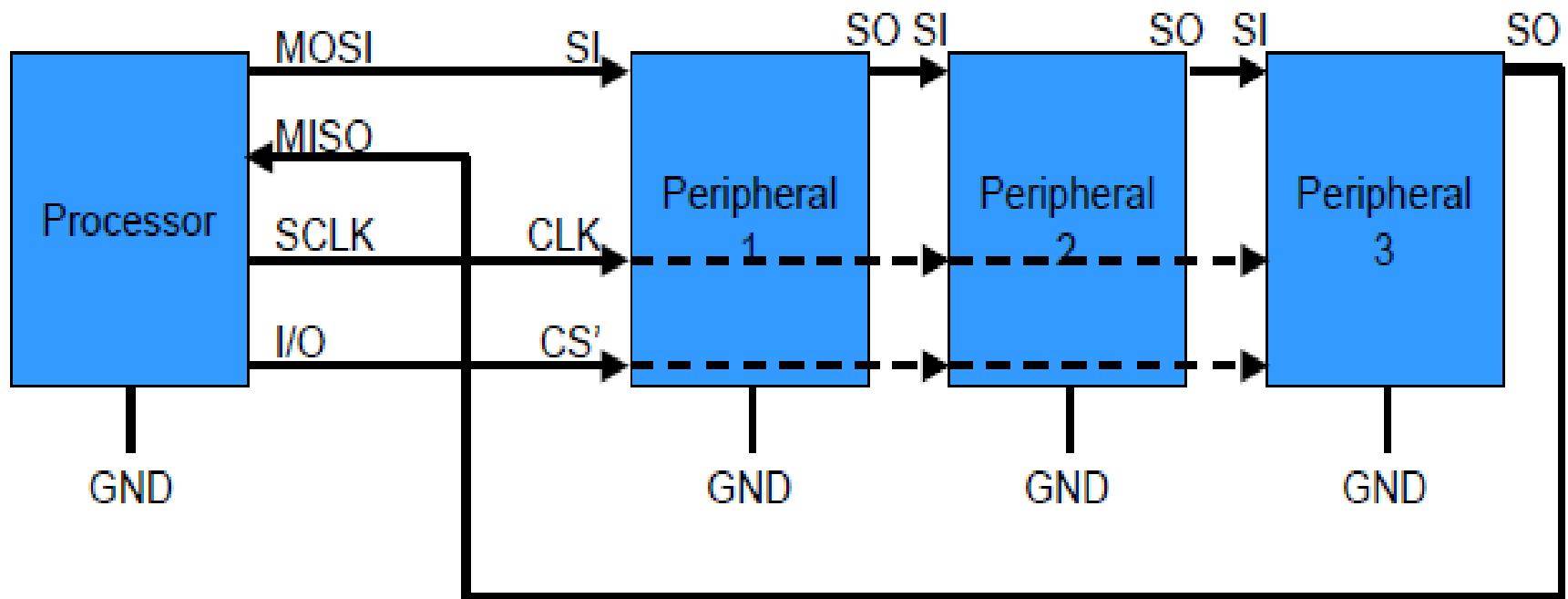
SPI Protocol



SPI Protocol



SPI Protocol

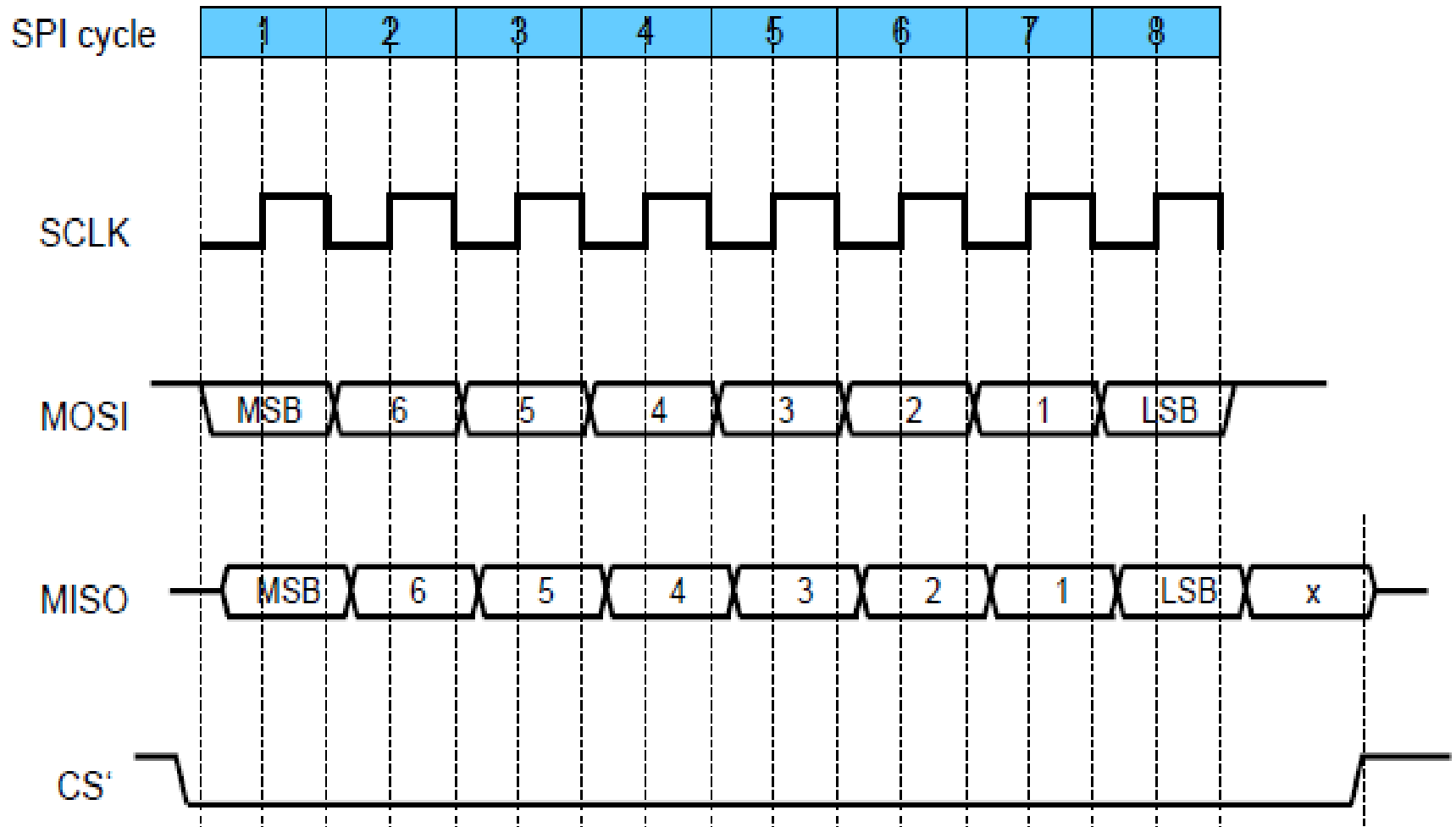


Extension to multiple peripherals:

- Real Time Clocks (time of day)
- Sensors (e.g. potentiometers)
- FLASH memory

- Interface speed limited by device technology
Mb/s, compared to kb/s for RS-232

SPI timing (Clock low, Clock phase 0)



Comparison (I2C & SPI)

Parameter	SPI	I2C
1. Maximum bit rate	Up to 10 MHz range or more	About 100 KHz (Standard) 400 KHz (Fast mode) 3.4 MHz (High Speed mode)
2. Interface pins required	Three plus one chip select per peripheral	Two, regardless number of peripherals
3. Number of devices sharing a bus	As many chip select pins are available	Bus can be address upto 127 devices
4. Point to point or multipoint	More efficient in point to point communication	More efficient in multi master, multi slave communication
5. Acknowledgement mechanism	Does not have Acknowledgement mechanism	Have an acknowledgement mechanism
6. Device addressing	Lack of built in device addressing	Built in device addressing and straight forward.

References:

- Textbooks prescribed in the university syllabus.
- As per the presentation slides received during Online Faculty Orientation Workshop under the aegis of BOS (E&TC), SPPU, Pune for syllabus discussions.

